Experiment	*Lab 3: AFL – Lift and Drag on Airfoils*
Author(s)	Clancy Crawford
Date	03/08/2026
Section Number	*15 Pratik Deshpande*

Summary

Lift and Drag on Airfoils lab (AFL) measured the pressure distribution and wake velocity profile of a NACA 4412 airfoil (6.0in chord) in a wind tunnel at multiple angles of attack ranging from -10° to 15° in increments of 5°. The surface pressure was measured to find the experiment's pressure coefficient distribution, lift and moment coefficients. A wake profile was also found to determine drag. Measured values include T = 21℃ ± 0.5℃ and P = 30.25$inHg$ ± 0.025$inHg$. The experiment's results were compared to predictions using panel method code that was provided. Lift coefficient vs Angle of Attack (AoA) increased linearly until about 5°, had a maximum of about 1.31 at 10°, then decrease until 15° due to flow separation. Drag coefficients ranged from the lowest at 0.0084 at -5° to a maximum of 0.25 at 15°, thus showing wake growth near stall. The zero-lift AoA = -7.29°. The experiment results showed close agreement with the given predictions for lift and moment coefficients at low AoA's, however the higher the AoA grew, deviations occurred due to flow separation.

Aim & Objectives

The aim of this experiment was to determine the aerodynamic characteristics of a 6in chord on a NACA 4412 airfoil, including lift, drag, and moment of coefficients for a range of angles of attack. Objectives include measuring the pressure distribution at multiple AoA's, finding the Cp distribution, lift, wake velocity profiles, drag, zero-lift AoA, and comparing coefficients using given panel code.

Uncertainty Analysis

The method used to calculate uncertainty is perturbation. Each value found was recalculated with the absolute value of one input at a time with its uncertainty and added to the total value. Measured values include $T = 21℃ \pm 0.5℃$, $P = 30.25inHg \pm 0.025inHg$. *Air density was found with the equation* $\rho = p/(RT)$, *where p and T were perturbed separately.*

Freestream velocity was calculated from the equation $V = \sqrt{\frac{2\Delta p}{\rho}}$ *and pressure coefficient with* $C_p = \frac{\Delta p}{q}$, *where q and ρ were perturbed and summed respectively. Change in differential pressure (q) was found with a 95% (1.96 standard deviations from the mean) confidence interval of the mean of data samples subtracted by the tare. Lift coefficients were found by integrating the difference between upper and lower pressure coefficients along the airfoil. Uncertainty was found by perturbing the upper and lower pressure coefficients and integrating.*

Drag was calculated by $D' = \rho \int_{wake\ begin}^{wake\ end} U_w(U_\infty - U_w)dz$ *with the perturbation of* ρ, U_w, U_∞ *, and wake position. Next, drag coefficient was computed by* $C_d = \frac{D'}{\frac{1}{2}\rho {U_\infty}^2 c}$*, with* $D', \rho, \text{and } U_\infty$ *perturbed. All computations were done in MATLAB (Appendix A).*

Table 1 shows ambient conditions and relevant values.

Table 1: Uncertainty and relevant values.

Variable	***Value***	***Uncertainty***
T	*21°C*	*±0.5°C*
P	*30.25 inHg*	*±0.025 inHg*
ρ	1.19 kg/m^3	*±0.02* kg/m^3
U_∞	14.9 m/s	*±0.3 m/s*
z	--	*±0.001 in*

Results & Discussion

Wake Velocity Profile

Figure 1 below shows the measured wake velocity profiles for the 6 in chord NACA 4412 airfoil at angles of attack ranging from -10° to 15° in 5° intervals. At smaller angles of attack, the velocity deficit and wake profile is smaller in comparison to larger angles of attack, like 15°, in which there is a larger wake profile. This indicated increased drag at 15° due to flow separation and turbulence. Drag causes the velocity deficit behind the airfoil. Uncertainty bars are shown for each measurement and derived from density and pressure then propagated.

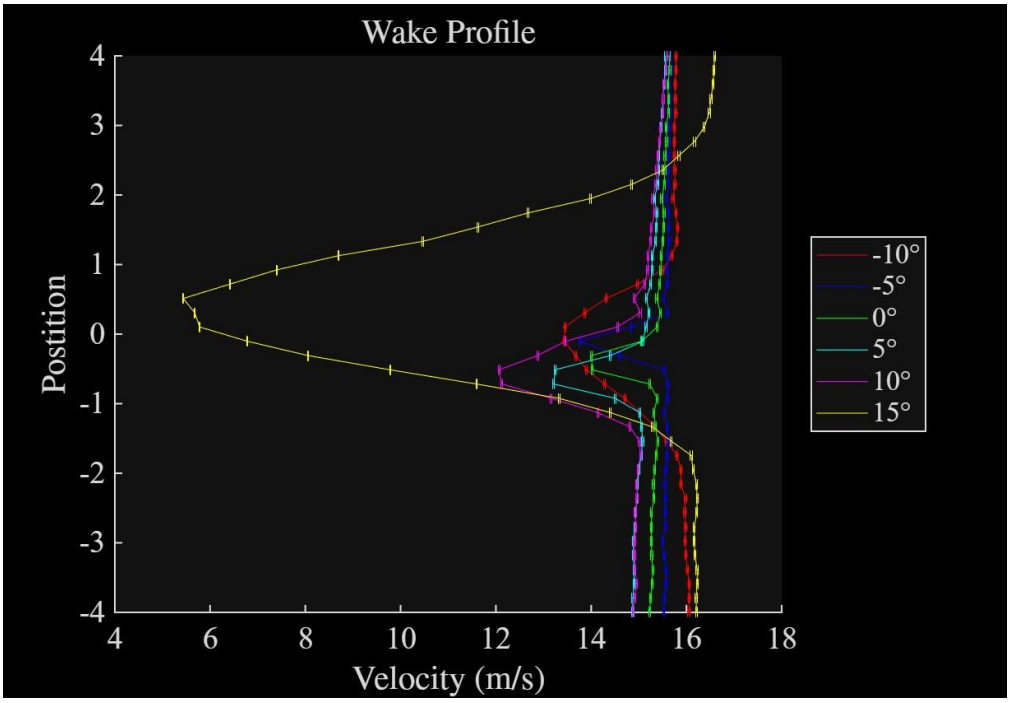


Figure 1: Wake Velocity Profile for each Angle of Attack.

Pressure Coefficient Distributions

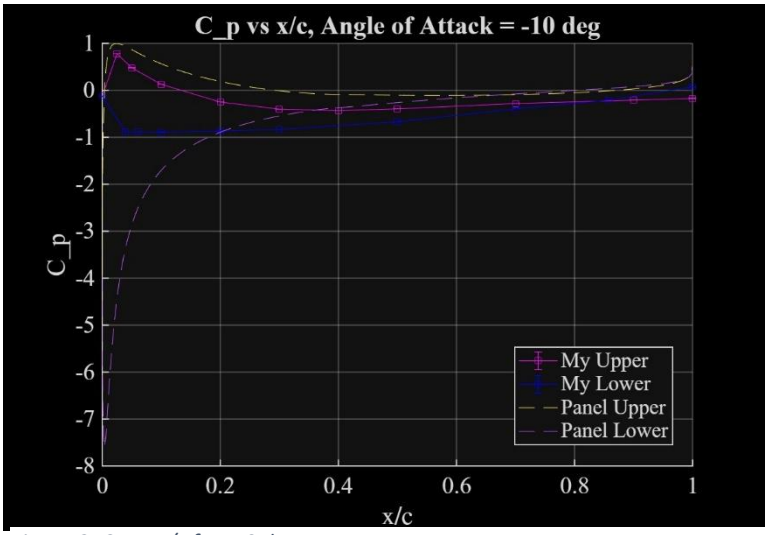


Figure 2: Cp vs x/c for -10 degrees.

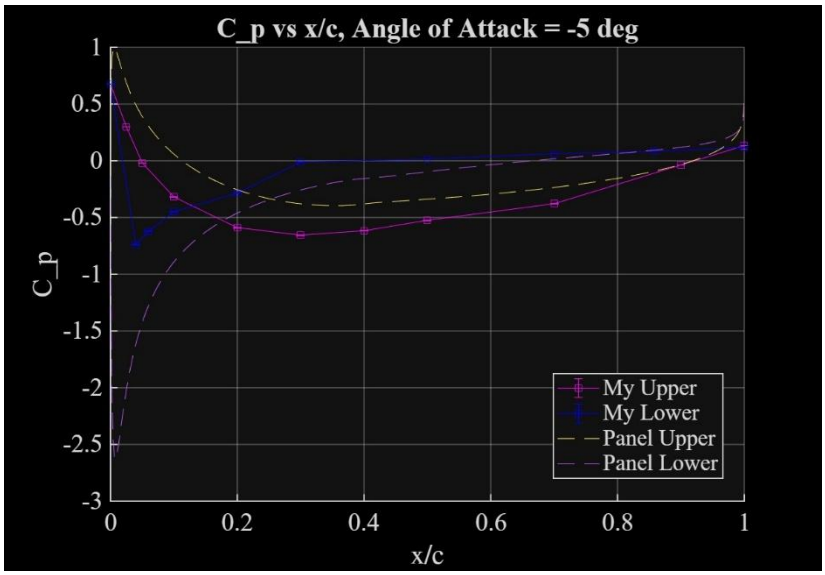


Figure 3: Cp vs x/c for -5 degrees.

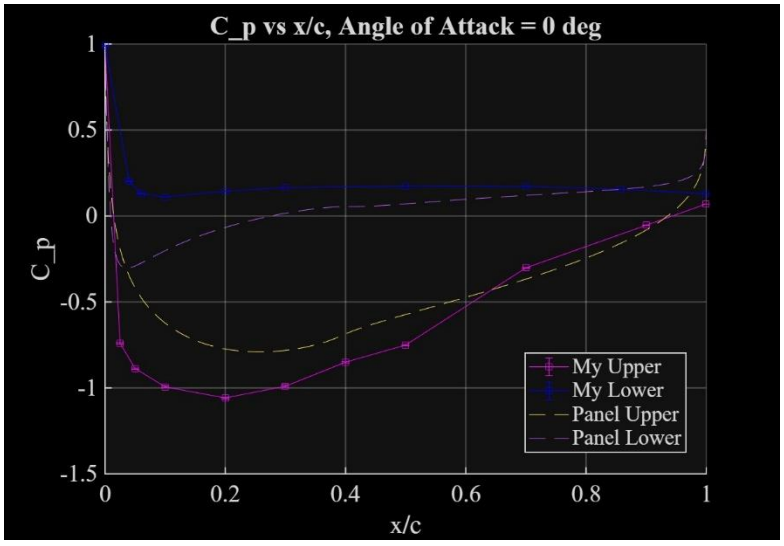


Figure 4: Cp vs x/c for 0 degrees.

Figure 2 – 7 shows the experiment's pressure distribution at each angle of attack and the panel code prediction. The pressure distributions show the behavior expected for this airfoil. At positive AoA's the upper surface shows lower pressure, and the lower surface shows higher pressure. This pressure difference produces lift. The Cp value at x/c = 1 was calculated by linearly extrapolating the end two pressure ports on the upper and lower surfaces. The extrapolated upper and lower values were averaged to define a single trailing-edge Cp used for both surfaces.

Near the leading edge, the magnitude increases as the AoA increases, corresponding to lift. At higher AoA's, like 15°, the experiment pressure starts to deviate from the panel predictions. This follows the idea as the panel prediction assumes inviscid flow, while the experiment undergoes real world viscous effects and flow separation. Uncertainty bars are shown for the experiment pressure values and were propagated from the pressure measurements and freestream pressure.

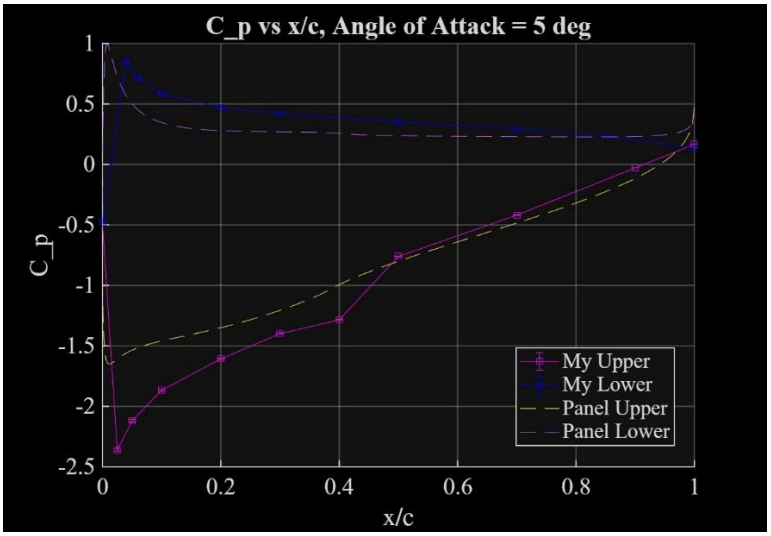


Figure 2: Cp vs x/c for 5 degrees.

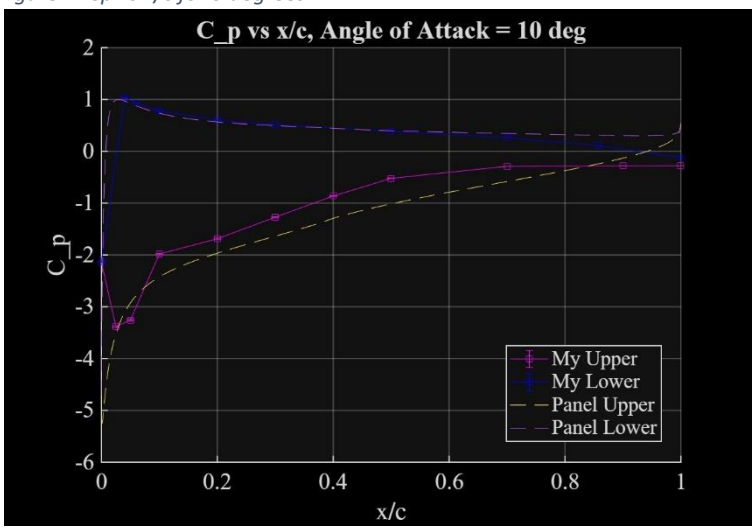


Figure 3: Cp vs x/c for 10 degrees.

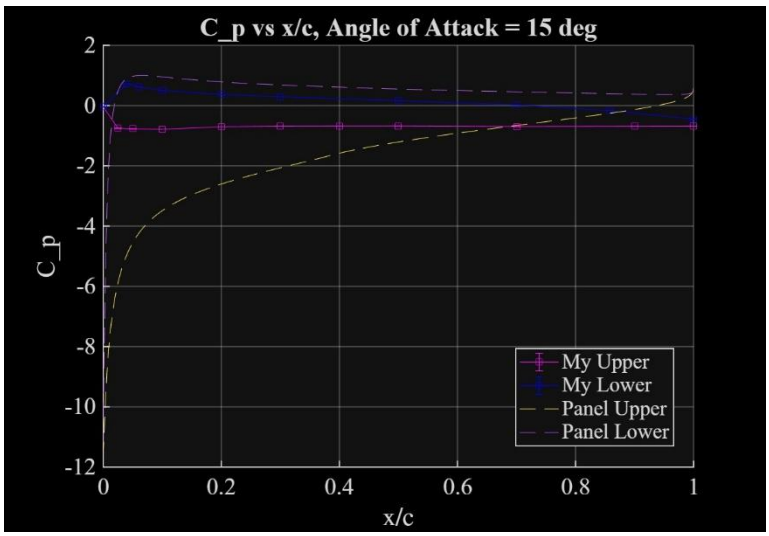


Figure 4: Cp vs x/c for 15 degrees.

Lift Coefficient vs. Angle of Attack

Figure 8 shows the experiment's lift coefficient in comparison to the panel method code. Consistent with thin airfoil theory, the experiment's lift coefficient increases linearly from -10° to approximately 5°. The maximum lift coefficient occurs at about 10° equaling approximately 1.31. After 10°, the lift coefficient decreases, showcasing stall and flow separation.

The panel method code predicts a linear increase in coefficient of lift for alpha, differing from the experiment's results for high angles of attack. Potential flow method neglects stalling effects because it does not allow for viscous effects.

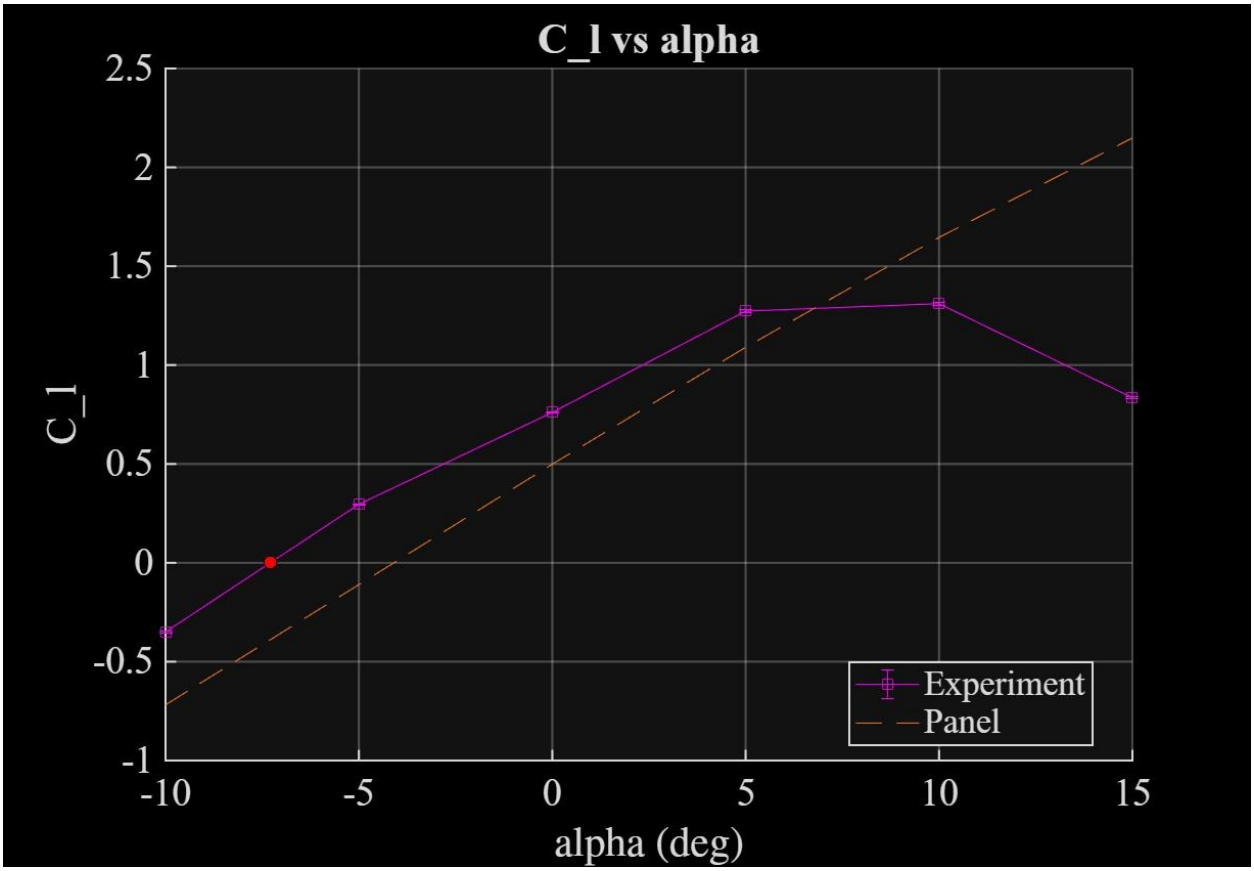


Figure 5: Lift Coefficient vs. Alpha for the experiment and panel code.

Drag Coefficient vs. Angle of Attack

Figure 9 below shows the drag coefficient from the wake momentum method. At lower angles of attack (-10° to 10°), the drag coefficient is relatively small. As the angle increases to 15°, the drag coefficient increases to approximately 0.25, corresponding to the wake velocity profile. In comparison, the panel code is shown by the potential flow at 0 drag coefficient due to inviscid flow making drag equal to zero.

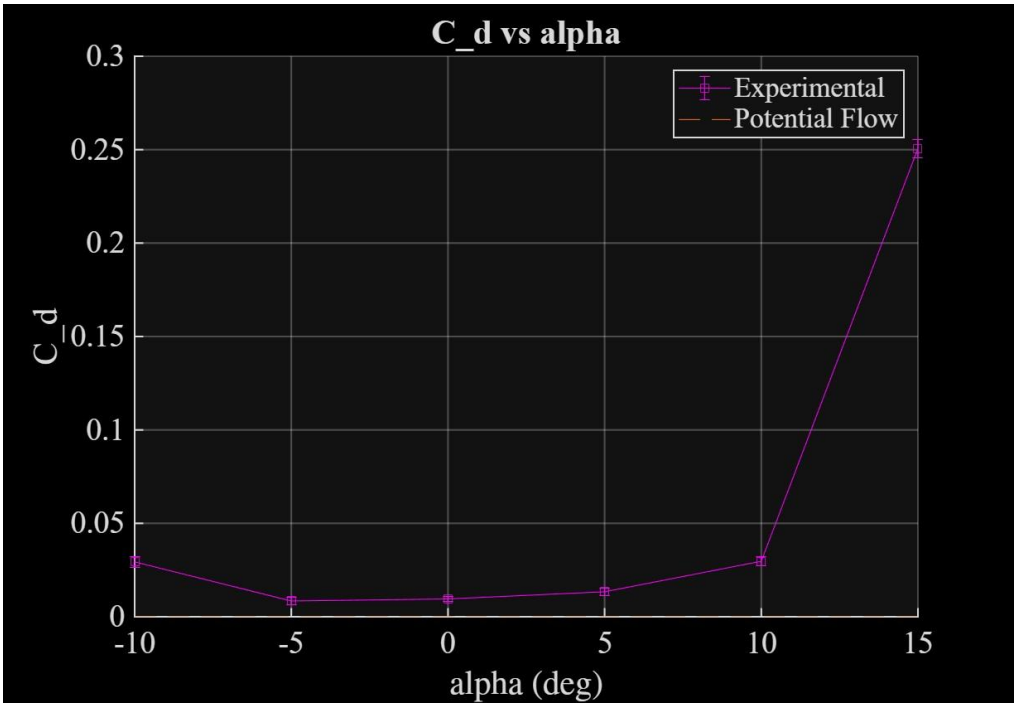


Figure 6: Drag Coefficient versus angle of attack for experimental data.

Drag Polar

Figure 10 shows the relationship between the coefficients of drag and lift for the experiment's airfoil. Drag slowly increases at moderate lift coefficients then has a sharp increase in drag at higher lift coefficients near stall conditions. Uncertainty bars are included for both lift and drag and represent calculated uncertainty from pressure and velocity.

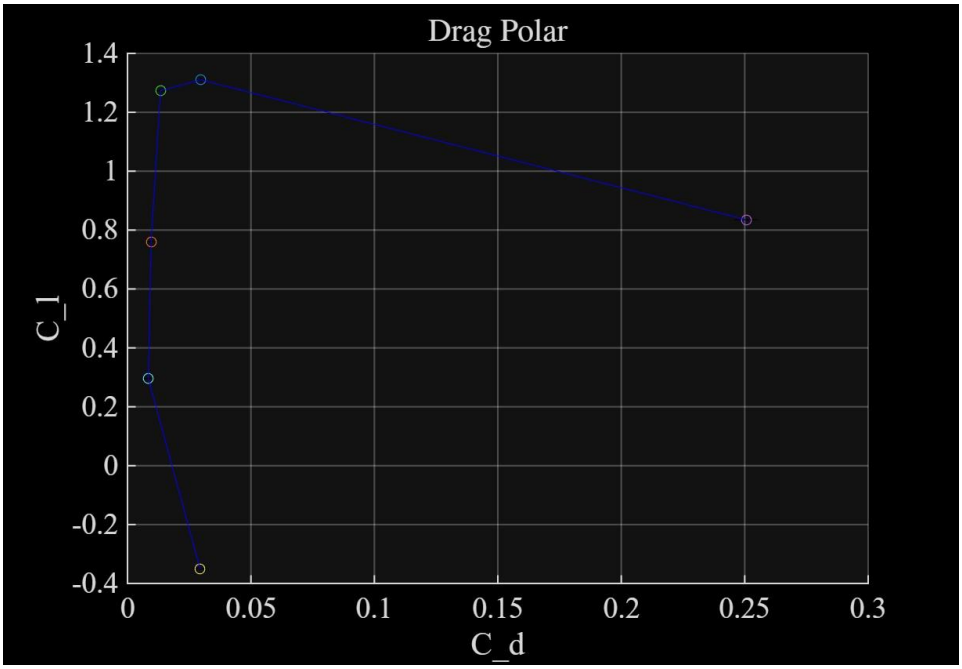


Figure 7: Comparing the coefficient of drag to the coefficient of lift for the experiment.

Lift/Drag Ratio

Figure 11 highlights the lift to drag ratio for the angles of attack of the experiment. Maximum efficiency occurs at approximately 5°, producing high lift and low amounts of drag. At the lower/higher angles of attack the efficiency increases/decreases rapidly due to the increase in drag because of flow separation.

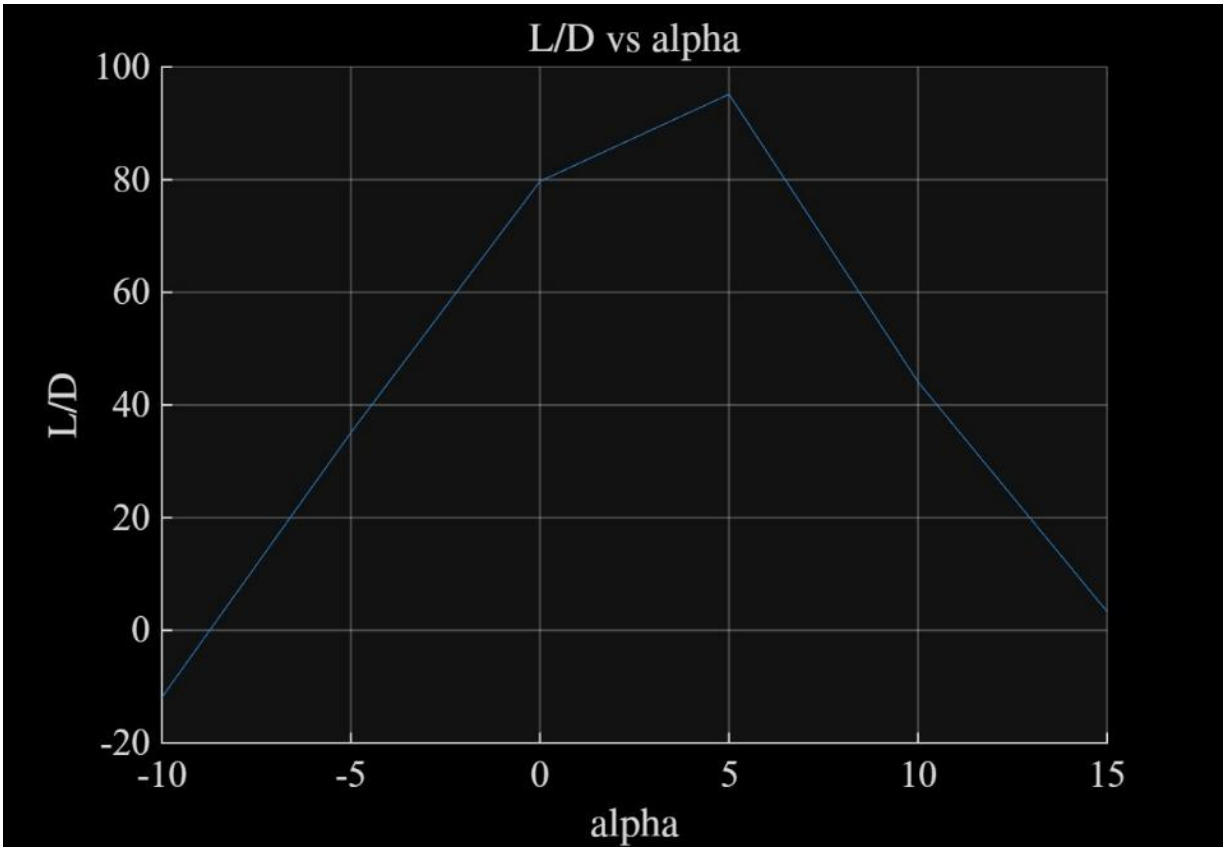


Figure 8: Lift to Drag ratio for the angles of attack.

Moment Coefficients

Figures 12 and 13 show the moment coefficient about the leading edge and quarter chord respectively. The leading-edge moment coefficient decreases as the angle of attack increases, due to increased suction on the upper surface. The experiment line follows with the panel's code until about 5° to 15°, this deviation is due to the panel assumes inviscid flow.

The quarter chord moment coefficient (Figure 13) experimental values are not as close to the expected panel code as shown on the graph near -10° and 10° but are within the expected range otherwise. The panel method shows an almost constant negative moment from -0.1 to -0.12. The deviation from panel code to experimental data are likely due to the viscous effects and flow separation that are not accounted for in the panel method.

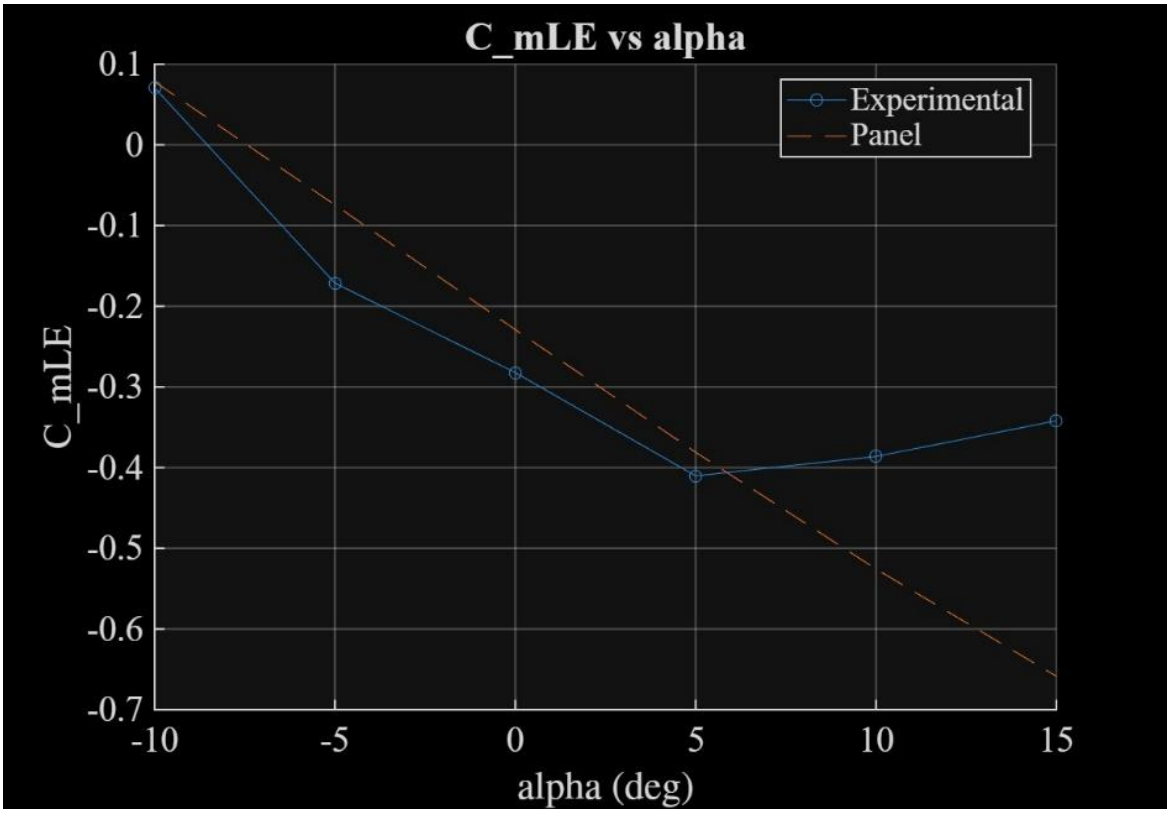


Figure 9: Coefficient of moment about the leading edge for angles of attack for both the experiment and panel method.

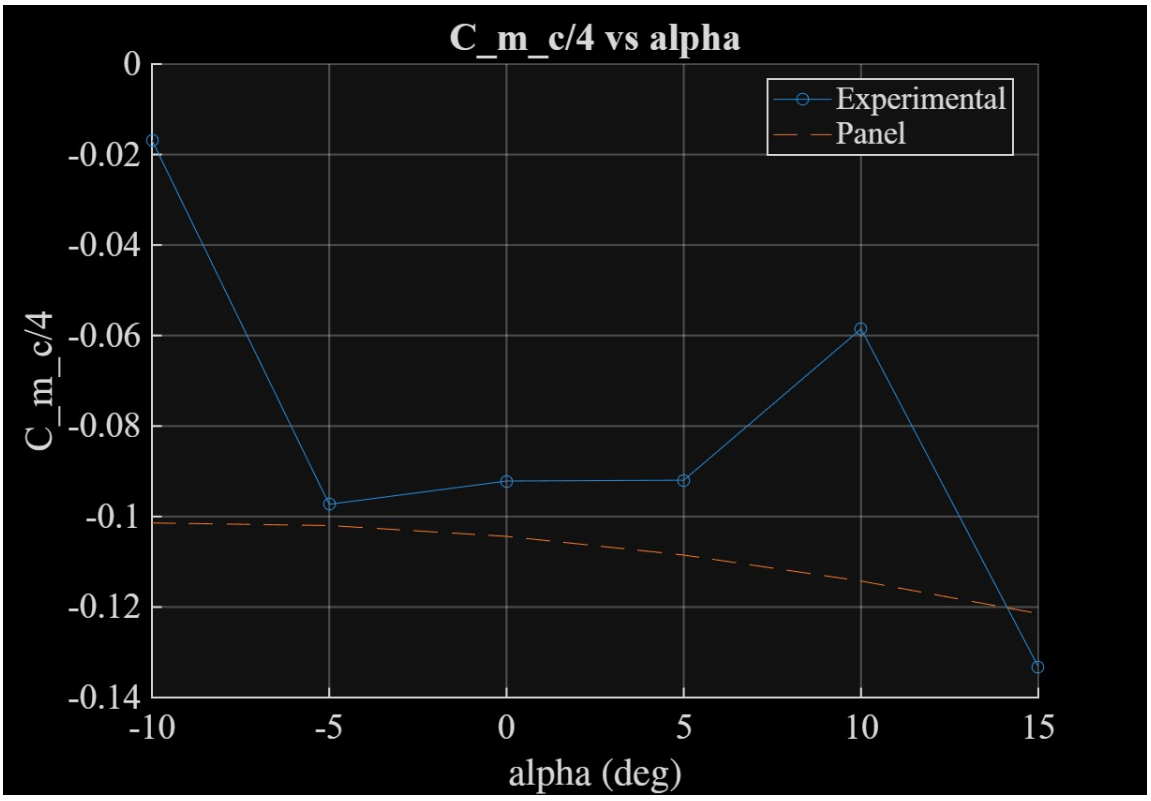


Figure 10: Coefficient of moment about a quarter of the chord versus alpha for both the experiment and panel method.

Zero-lift Angle of Attack

The zero-lift angle of attack from the experimental data was calculated by interpolating the experimental lift coefficients that are nearest to zero lift. The zero-lift angle equaled -7.29° (as shown in Figure 8 by the red marker).

As published in NASA technical memorandum 19930090976, the zero-lift angle of attack is approximately -4°. The difference between the experiment and published data can be attributed to viscous effects, Reynolds number differences, uncertainty calculations, experiment differences and various other effects.

References

Abbott, I. H., Von Doenhoff, A. E. & Stivers, L. Summary of Airfoil Data. NASA NTRS (1945). Available at: https://ntrs.nasa.gov/api/citations/19930090976/downloads/19930090976.pdf. (Accessed: 10th March 2026)

AE315_INT_ (01.30.26).pdf

AFL Lab Processing Handout.PDF

Appendix

```
% AE315 DB15 Group 15.2: Lab 3
% Clancy Crawford   Student ID:2654296
% 2/24/2026

clear;
close all;
clc;

% Givens
T_atm = 21; % degrees C
T_atm = T_atm + 273.15; % K
T_u = 0.5; % K - half the smallest scale division: 1 deg C
R = 287; % J/(kg*k)
P_atm = 30.25; % inHg
P_atm = P_atm * 3386.39; % Pa
P_u = 0.025* 3386.39;

% finding rho with uncertainty
rho = P_atm/(R*T_atm); % kg/m^3

rho_P = (P_atm + P_u)/(R*T_atm);
abs_rhoP = abs(rho_P - rho);

rho_T = P_atm/(R*(T_atm + T_u));
abs_rhoT = abs(rho_T - rho);

rho_u = abs_rhoT + abs_rhoP;
```

```
% data
tare = load("Tare_20260217T084939.mat").rawData(:,1);
dp_neg10 = load("neg10_20260217T085911.mat").rawData(:,:);
dp_neg5 = load('neg5_20260217T090746.mat').rawData(:,:);
dp_0 = load("zero_20260217T091716.mat").rawData(:,:);
dp_5 = load("5_20260217T092816.mat").rawData(:,:);
dp_10 = load('10_20260217T093713.mat').rawData(:,:);
dp_15 = load("15_20260217T094553.mat").rawData(:,:);

% finding v and its uncertanity
tare_mean = mean(tare, 2);

dp_neg10 = dp_neg10 - tare_mean;
dp_neg5 = dp_neg5 - tare_mean;
dp_0 = dp_0 - tare_mean;
dp_5 = dp_5 - tare_mean;
dp_10 = dp_10 - tare_mean;
dp_15 = dp_15 - tare_mean;

dp_neg10_mean = mean(dp_neg10, 2)*248.84; %Pa
dp_neg5_mean = mean(dp_neg5, 2)*248.84; %Pa
dp_0_mean = mean(dp_0, 2)*248.84; %Pa
dp_5_mean = mean(dp_5, 2)*248.84; %Pa
dp_10_mean = mean(dp_10, 2)*248.84; %Pa
dp_15_mean = mean(dp_15, 2)*248.84; %Pa

v_mean_neg10 = sqrt((2*dp_neg10_mean)/rho);
v_mean_neg5 = sqrt((2*dp_neg5_mean)/rho);
v_mean_0 = sqrt((2*dp_0_mean)/rho);
v_mean_5 = sqrt((2*dp_5_mean)/rho);
v_mean_10 = sqrt((2*dp_10_mean)/rho);
v_mean_15 = sqrt((2*dp_15_mean)/rho);

v_mean = {v_mean_neg10, v_mean_neg5, v_mean_0, v_mean_5, v_mean_10, v_mean_15};

v_mean_neg10_rho_u = sqrt((2*dp_neg10_mean)./(rho_u+rho));
v_mean_neg10_u = v_mean_neg10_rho_u - v_mean_neg10;
v_mean_neg5_rho_u = sqrt((2*dp_neg5_mean)./(rho_u+rho));
v_mean_neg5_u = v_mean_neg5_rho_u - v_mean_neg5;
v_mean_0_rho_u = sqrt((2*dp_0_mean)./(rho_u+rho));
v_mean_0_u = v_mean_0_rho_u - v_mean_0;
v_mean_5_rho_u = sqrt((2*dp_5_mean)./(rho_u+rho));
v_mean_5_u = v_mean_5_rho_u - v_mean_5;
v_mean_10_rho_u = sqrt((2*dp_10_mean)./(rho_u+rho));
v_mean_10_u = v_mean_10_rho_u - v_mean_10;
v_mean_15_rho_u = sqrt((2*dp_15_mean)./(rho_u+rho));
v_mean_15_u = v_mean_15_rho_u - v_mean_15;

v_mean_u = {abs(v_mean_neg10_u), abs(v_mean_neg5_u), abs(v_mean_0_u), abs(v_mean_5_u),
abs(v_mean_10_u), abs(v_mean_15_u)};

% plotting wake profile
```

```
z = linspace(-4, 4, 40); % in
z_m = z * 0.0254; % m
z_u = 0.0001 * 0.0254; % uncertanity

figure()
hold on
plot(v_mean_neg10, z, 'r')
plot(v_mean_neg5, z, 'b')
plot(v_mean_0, z, 'g')
plot(v_mean_5, z, 'c')
plot(v_mean_10, z, 'm')
plot(v_mean_15, z, 'y')
errorbar(v_mean_neg10, z, v_mean_neg10_u, 'horizontal', 'r.-')
errorbar(v_mean_neg5, z, v_mean_neg5_u, 'horizontal', 'b')
errorbar(v_mean_0, z, v_mean_0_u, 'horizontal', 'g')
errorbar(v_mean_5, z, v_mean_5_u, 'horizontal', 'c')
errorbar(v_mean_10, z, v_mean_10_u, 'horizontal', 'm')
errorbar(v_mean_15, z, v_mean_15_u, 'horizontal', 'y')
hold off
title('Wake Profile')
xlabel('Velocity (m/s)')
ylabel('Postition')
legend('-10°','-5°','0°','5°','10°','15°','Location', 'east outside')


% limits found using this input method
%[p1 p2] = ginput(2);
%[~,iz1] = min(abs(z-p2(1)));
%[~,iz2] = min(abs(z-p2(2)));
%iz1_15 = iz1;
%iz2_15 = iz2;
%save('V_15_wake_limits',"iz1_15","iz2_15")

%loading limits
ll_neg10 = load('V_neg10_wake_limits.mat');
iz1_neg10 = ll_neg10.iz1_neg10;
iz2_neg10 = ll_neg10.iz2_neg10;
ll_neg5 = load('V_neg5_wake_limits.mat');
iz1_neg5 = ll_neg5.iz1_neg5;
iz2_neg5 = ll_neg5.iz2_neg5;
ll_0 = load('V_0_wake_limits.mat');
iz1_0 = ll_0.iz1_0;
iz2_0 = ll_0.iz2_0;
ll_5 = load('V_5_wake_limits.mat');
iz1_5 = ll_5.iz1_5;
iz2_5 = ll_5.iz2_5;
ll_10 = load('V_10_wake_limits.mat');
iz1_10 = ll_10.iz1_10;
iz2_10 = ll_10.iz2_10;
ll_15 = load('V_15_wake_limits.mat');
iz1_15 = ll_15.iz1_15;
iz2_15 = ll_15.iz2_15;
```

```
iz1 = [iz1_neg10, iz1_neg5, iz1_0, iz1_5, iz1_10, iz1_15];
iz2 = [iz2_neg10, iz2_neg5, iz2_0, iz2_5, iz2_10, iz2_15];

alpha = [-10, -5, 0, 5, 10, 15];

for i = 1:length(alpha)
    start_i = min(iz1(i), iz2(i));
    end_i   = max(iz1(i), iz2(i));
    iz1(i) = start_i;
    iz2(i) = end_i;
end


% finding u_inf and uncertanity
u_neg10 = v_mean_neg10;
u_top_neg10 = mean(u_neg10(1:iz1_neg10));
u_bottom_neg10 = mean(u_neg10(iz2_neg10:end));
u_inf_neg10 = 0.5*(u_top_neg10 + u_bottom_neg10);
u_neg5 = v_mean_neg5;
u_top_neg5 = mean(u_neg5(1:iz1_neg5));
u_bottom_neg5 = mean(u_neg5(iz2_neg5:end));
u_inf_neg5 = 0.5*(u_top_neg5 + u_bottom_neg5);
u_0 = v_mean_0;
u_top_0 = mean(u_0(1:iz1_0));
u_bottom_0 = mean(u_0(iz2_0:end));
u_inf_0 = 0.5*(u_top_0 + u_bottom_0);
u_5 = v_mean_5;
u_top_5 = mean(u_5(1:iz1_5));
u_bottom_5 = mean(u_5(iz2_5:end));
u_inf_5 = 0.5*(u_top_5 + u_bottom_5);
u_10 = v_mean_10;
u_top_10 = mean(u_10(1:iz1_10));
u_bottom_10 = mean(u_10(iz2_10:end));
u_inf_10 = 0.5*(u_top_10 + u_bottom_10);
u_15 = v_mean_15;
u_top_15 = mean(u_15(1:iz1_15));
u_bottom_15 = mean(u_15(iz2_15:end));
u_inf_15 = 0.5*(u_top_15 + u_bottom_15);

u_inf = [u_inf_neg10, u_inf_neg5, u_inf_0, u_inf_5, u_inf_10, u_inf_15];

u_inf_u_neg10 = 0.5*( mean(abs(v_mean_neg10_u(1:iz1_neg10))) +
mean(abs(v_mean_neg10_u(iz2_neg10:end))) );
u_inf_u_neg5 = 0.5*( mean(abs(v_mean_neg5_u(1:iz1_neg5))) +
mean(abs(v_mean_neg5_u(iz2_neg5:end))) );
u_inf_u_0 = 0.5*( mean(abs(v_mean_0_u(1:iz1_0))) + mean(abs(v_mean_0_u(iz2_0:end))) );
u_inf_u_5 = 0.5*( mean(abs(v_mean_5_u(1:iz1_5))) + mean(abs(v_mean_5_u(iz2_5:end))) );
u_inf_u_10 = 0.5*( mean(abs(v_mean_10_u(1:iz1_10))) + mean(abs(v_mean_10_u(iz2_10:end))) );
u_inf_u_15 = 0.5*( mean(abs(v_mean_15_u(1:iz1_15))) + mean(abs(v_mean_15_u(iz2_15:end))) );

u_inf_u = [abs(u_inf_u_neg10), abs(u_inf_u_neg5), abs(u_inf_u_0), abs(u_inf_u_5),
abs(u_inf_u_10), abs(u_inf_u_15)];
```

```
%vinf = avg(v(1 through 11) and avg(v(27 through 40)

% finding q_inf and uncertanity
q_inf_neg10 = 0.5 * rho * u_inf_neg10^2;
q_inf_neg5 = 0.5 * rho * u_inf_neg5^2;
q_inf_0 = 0.5 * rho * u_inf_0^2;
q_inf_5 = 0.5 * rho * u_inf_5^2;
q_inf_10 = 0.5 * rho * u_inf_10^2;
q_inf_15 = 0.5 * rho * u_inf_15^2;

q_inf_u_rho_neg10 = abs(0.5*(rho+rho_u)*u_inf_neg10^2 - q_inf_neg10);
q_inf_u_rho_neg5 = abs(0.5*(rho+rho_u)*u_inf_neg5^2 - q_inf_neg5);
q_inf_u_rho_0 = abs(0.5*(rho+rho_u)*u_inf_0^2 - q_inf_0);
q_inf_u_rho_5 = abs(0.5*(rho+rho_u)*u_inf_5^2 - q_inf_5);
q_inf_u_rho_10 = abs(0.5*(rho+rho_u)*u_inf_10^2 - q_inf_10);
q_inf_u_rho_15 = abs(0.5*(rho+rho_u)*u_inf_15^2 - q_inf_15);

q_inf_u_u_inf_neg10 = abs( 0.5*rho*(u_inf_neg10 + u_inf_u_neg10)^2 - q_inf_neg10 );
q_inf_u_u_inf_neg5 = abs( 0.5*rho*(u_inf_neg5 + u_inf_u_neg5)^2 - q_inf_neg5);
q_inf_u_u_inf_0 = abs( 0.5*rho*(u_inf_0 + u_inf_u_0)^2 - q_inf_0);
q_inf_u_u_inf_5 = abs( 0.5*rho*(u_inf_5 + u_inf_u_5)^2 - q_inf_5);
q_inf_u_u_inf_10 = abs( 0.5*rho*(u_inf_10 + u_inf_u_10)^2 - q_inf_10);
q_inf_u_u_inf_15 = abs( 0.5*rho*(u_inf_15 + u_inf_u_15)^2 - q_inf_15);

q_inf_u_neg10 = q_inf_u_rho_neg10 + q_inf_u_u_inf_neg10;
q_inf_u_neg5 = q_inf_u_rho_neg5 + q_inf_u_u_inf_neg5;
q_inf_u_0 = q_inf_u_rho_0 + q_inf_u_u_inf_0;
q_inf_u_5 = q_inf_u_rho_5 + q_inf_u_u_inf_5;
q_inf_u_10 = q_inf_u_rho_10 + q_inf_u_u_inf_10;
q_inf_u_15 = q_inf_u_rho_15 + q_inf_u_u_inf_15;

% finding Cp
% loading port pressure data from col 3:20 and converting from H20 to Pa
port_neg10 = readmatrix('neg10_15ms.csv');
port_neg5 = readmatrix('neg5_15ms.csv');
port_0 = readmatrix('zero_15ms.csv');
port_5 = readmatrix('5_15ms.csv');
port_10 = readmatrix('10_15ms.csv');
port_15 = readmatrix('15_15ms.csv');
port_neg10 = (port_neg10(:, 3:20))*248.84; %Pa
port_neg5 = (port_neg5(:, 3:20))*248.84; %Pa
port_0 = (port_0(:, 3:20))*248.84; %Pa
port_5 = (port_5(:, 3:20))*248.84; %Pa
port_10 = (port_10(:, 3:20))*248.84; %Pa
port_15 = (port_15(:, 3:20))*248.84; %Pa

% finding mean of port and std
port_mean_neg10 = mean(port_neg10, 1);
port_mean_neg5 = mean(port_neg5, 1);
port_mean_0 = mean(port_0, 1);
port_mean_5 = mean(port_5, 1);
port_mean_10 = mean(port_10, 1);
```

```
port_mean_15 = mean(port_15, 1);

%QUESTION: which one am i supposed to use - the one i am or this method
%N = size(port_neg10,1);
%port_std_neg10 = std(port_neg10,0,1);
%p_std_u_neg10 = 1.96 * port_std_neg10 / sqrt(N);

port_std_neg10 = std(port_mean_neg10, 1, 1);
port_std_neg5 = std(port_mean_neg5, 1, 1);
port_std_0 = std(port_mean_0, 1, 1);
port_std_5 = std(port_mean_5, 1, 1);
port_std_10 = std(port_mean_10, 1, 1);
port_std_15 = std(port_mean_15, 1, 1);

p_std_u_neg10 = (1.96 * port_std_neg10) / sqrt(size(port_neg10, 1));
p_std_u_neg5 = (1.96 * port_std_neg5) / sqrt(size(port_neg5, 1));
p_std_u_0 = (1.96 * port_std_0) / sqrt(size(port_0, 1));
p_std_u_5 = (1.96 * port_std_5) / sqrt(size(port_5, 1));
p_std_u_10 = (1.96 * port_std_10) / sqrt(size(port_10, 1));
p_std_u_15 = (1.96 * port_std_15) / sqrt(size(port_15, 1));


% finding Cp with uncertanity
cp_neg10 = port_mean_neg10 ./ q_inf_neg10;
cp_neg5 = port_mean_neg5 ./ q_inf_neg5;
cp_0 = port_mean_0 ./ q_inf_0;
cp_5 = port_mean_5 ./ q_inf_5;
cp_10 = port_mean_10 ./ q_inf_10;
cp_15 = port_mean_15 ./ q_inf_15;

cp_u_p_neg10 = p_std_u_neg10 ./ q_inf_neg10;
cp_u_p_neg5 = p_std_u_neg5 ./ q_inf_neg5;
cp_u_p_0 = p_std_u_0 ./ q_inf_0;
cp_u_p_5 = p_std_u_5 ./ q_inf_5;
cp_u_p_10 = p_std_u_10 ./ q_inf_10;
cp_u_p_15 = p_std_u_15 ./ q_inf_15;

cp_u_q_neg10 = abs(port_mean_neg10 ./ (q_inf_neg10 + q_inf_u_neg10) - cp_neg10);
cp_u_q_neg5 = abs(port_mean_neg5 ./ (q_inf_neg5 + q_inf_u_neg5) - cp_neg5);
cp_u_q_0 = abs(port_mean_0 ./ (q_inf_0 + q_inf_u_0) - cp_0);
cp_u_q_5 = abs(port_mean_5 ./ (q_inf_5 + q_inf_u_5) - cp_5);
cp_u_q_10 = abs(port_mean_10 ./ (q_inf_10 + q_inf_u_10) - cp_10);
cp_u_q_15 = abs(port_mean_15 ./ (q_inf_15 + q_inf_u_15) - cp_15);

cp_u_neg10 = cp_u_p_neg10 + cp_u_q_neg10;
cp_u_neg5 = cp_u_p_neg5 + cp_u_q_neg5;
cp_u_0 = cp_u_p_0 + cp_u_q_0;
cp_u_5 = cp_u_p_5 + cp_u_q_5;
cp_u_10 = cp_u_p_10 + cp_u_q_10;
cp_u_15 = cp_u_p_15 + cp_u_q_15;

% port loactions for a NACA 4412 with a chord = 6.0 in
c = 6.0; % in
```

```
c_m = c * 0.0254;
u_x = [0, 0.1500, 0.3000, 0.6000, 1.2000, 1.7998, 2.4001, 3.0000, 4.2000, 5.4000];
l_x = [0, 0.2400, 0.3600, 0.6000, 1.200, 1.8000, 3.0000, 4.2000, 5.1500];

% splitting cp into upper and lower with uncertanity
cp_up_neg10 = cp_neg10(1:10);
cp_up_u_neg10 = cp_u_neg10(1:10);
cp_l_neg10 = [cp_neg10(1), cp_neg10(11:18)];
cp_l_u_neg10 = [cp_u_neg10(1), cp_u_neg10(11:18)];
cp_up_neg5 = cp_neg5(1:10);
cp_up_u_neg5 = cp_u_neg5(1:10);
cp_l_neg5 = [cp_neg5(1), cp_neg5(11:18)];
cp_l_u_neg5 = [cp_u_neg5(1), cp_u_neg5(11:18)];
cp_up_0 = cp_0(1:10);
cp_up_u_0 = cp_u_0(1:10);
cp_l_0 = [cp_0(1), cp_0(11:18)];
cp_l_u_0 = [cp_u_0(1), cp_u_0(11:18)];
cp_up_5 = cp_5(1:10);
cp_up_u_5 = cp_u_5(1:10);
cp_l_5 = [cp_5(1), cp_5(11:18)];
cp_l_u_5 = [cp_u_5(1), cp_u_5(11:18)];
cp_up_10 = cp_10(1:10);
cp_up_u_10 = cp_u_10(1:10);
cp_l_10 = [cp_10(1), cp_10(11:18)];
cp_l_u_10 = [cp_u_10(1), cp_u_10(11:18)];
cp_up_15 = cp_15(1:10);
cp_up_u_15 = cp_u_15(1:10);
cp_l_15 = [cp_15(1), cp_15(11:18)];
cp_l_u_15 = [cp_u_15(1), cp_u_15(11:18)];

% for trailing edge: x/c=1 @ upper: port 9 and 10, lower: 17 and 18
cp_te_up_neg10 = interp1(u_x(9:10), cp_up_neg10(9:10), 6.0, 'linear', 'extrap');
cp_te_l_neg10 = interp1(l_x(7:8), cp_l_neg10(8:9), 6.0, 'linear', 'extrap');
cp_te_u_neg10 = mean([cp_u_neg10(9), cp_u_neg10(10), cp_u_neg10(17), cp_u_neg10(18)]);

cp_te_up_neg5 = interp1(u_x(9:10), cp_up_neg5(9:10), 6.0, 'linear', 'extrap');
cp_te_l_neg5 = interp1(l_x(7:8), cp_l_neg5(8:9), 6.0, 'linear', 'extrap');
cp_te_u_neg5 = mean([cp_u_neg5(9), cp_u_neg5(10), cp_u_neg5(17), cp_u_neg5(18)]);

cp_te_up_0 = interp1(u_x(9:10), cp_up_0(9:10), 6.0, 'linear', 'extrap');
cp_te_l_0 = interp1(l_x(7:8), cp_l_0(8:9), 6.0, 'linear', 'extrap');
cp_te_u_0 = mean([cp_u_0(9), cp_u_0(10), cp_u_0(17), cp_u_0(18)]);

cp_te_up_5 = interp1(u_x(9:10), cp_up_5(9:10), 6.0, 'linear', 'extrap');
cp_te_l_5 = interp1(l_x(7:8), cp_l_5(8:9), 6.0, 'linear', 'extrap');
cp_te_u_5 = mean([cp_u_5(9), cp_u_5(10), cp_u_5(17), cp_u_5(18)]);

cp_te_up_10 = interp1(u_x(9:10), cp_up_10(9:10), 6.0, 'linear', 'extrap');
cp_te_l_10 = interp1(l_x(7:8), cp_l_10(8:9), 6.0, 'linear', 'extrap');
cp_te_u_10 = mean([cp_u_10(9), cp_u_10(10), cp_u_10(17), cp_u_10(18)]);

cp_te_up_15 = interp1(u_x(9:10), cp_up_15(9:10), 6.0, 'linear', 'extrap');
cp_te_l_15 = interp1(l_x(7:8), cp_l_15(8:9), 6.0, 'linear', 'extrap');
```

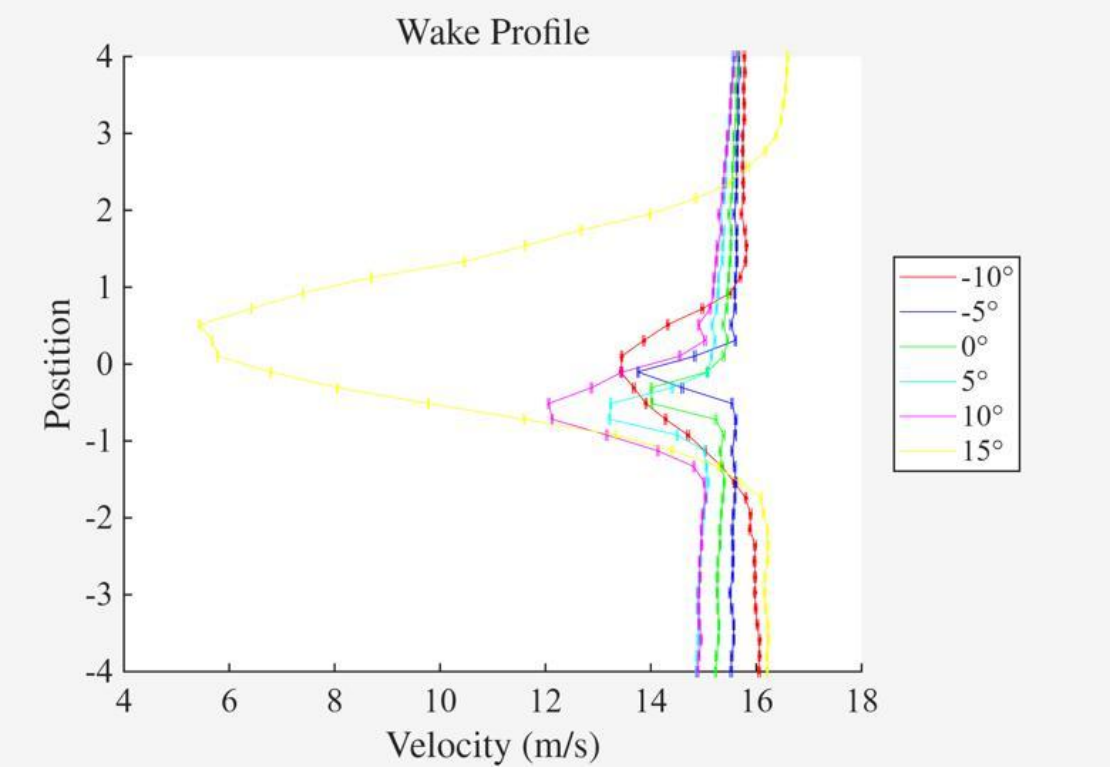
```
cp_te_u_15 = mean([cp_u_15(9), cp_u_15(10), cp_u_15(17), cp_u_15(18)]);

% putting trailing edge into the arrays for both upper and lower with
% uncertainty
up_x_te = [u_x/c, 1.0]; % in
cp_te_up_neg10 = [cp_up_neg10, cp_te_up_neg10];
cp_te_up_u_neg10 = [cp_up_u_neg10, cp_te_u_neg10];
cp_te_up_neg5 = [cp_up_neg5, cp_te_up_neg5];
cp_te_up_u_neg5 = [cp_up_u_neg5, cp_te_u_neg5];
cp_te_up_0 = [cp_up_0, cp_te_up_0];
cp_te_up_u_0 = [cp_up_u_0, cp_te_u_0];
cp_te_up_5 = [cp_up_5, cp_te_up_5];
cp_te_up_u_5 = [cp_up_u_5, cp_te_u_5];
cp_te_up_10 = [cp_up_10, cp_te_up_10];
cp_te_up_u_10 = [cp_up_u_10, cp_te_u_10];
cp_te_up_15 = [cp_up_15, cp_te_up_15];
cp_te_up_u_15 = [cp_up_u_15, cp_te_u_15];

l_x_te = [l_x/c, 1.0]; % in
cp_te_l_neg10 = [cp_l_neg10, cp_te_l_neg10];
cp_te_l_u_neg10 = [cp_l_u_neg10, cp_te_u_neg10];
cp_te_l_neg5 = [cp_l_neg5, cp_te_l_neg5];
cp_te_l_u_neg5 = [cp_l_u_neg5, cp_te_u_neg5];
cp_te_l_0 = [cp_l_0, cp_te_l_0];
cp_te_l_u_0 = [cp_l_u_0, cp_te_u_0];
cp_te_l_5 = [cp_l_5, cp_te_l_5];
cp_te_l_u_5 = [cp_l_u_5, cp_te_u_5];
cp_te_l_10 = [cp_l_10, cp_te_l_10];
cp_te_l_u_10 = [cp_l_u_10, cp_te_u_10];
cp_te_l_15 = [cp_l_15, cp_te_l_15];
cp_te_l_u_15 = [cp_l_u_15, cp_te_u_15];

% panel code
cp_u = cell(size(alpha));
cp_l = cell(size(alpha));
cl  = zeros(size(alpha));
cm_le = zeros(size(alpha));
cm_1_4 = zeros(size(alpha));

for i=1: length(alpha)
    [cp_u{i}, cp_l{i}, xu, xl, cl(i), cm_le(i), cm_1_4(i)] = panelcodefunc(4412, 201, alpha(i));
end

% overlaying the panel cp and my cp
cp_up = {cp_te_up_neg10, cp_te_up_neg5, cp_te_up_0, cp_te_up_5, cp_te_up_10, cp_te_up_15};
cp_low = {cp_te_l_neg10, cp_te_l_neg5, cp_te_l_0, cp_te_l_5, cp_te_l_10, cp_te_l_15};
cp_up_u = {cp_te_up_u_neg10, cp_te_up_u_neg5, cp_te_up_u_0, cp_te_up_u_5, cp_te_up_u_10,
cp_te_up_u_15};
cp_low_u = {cp_te_l_u_neg10, cp_te_l_u_neg5, cp_te_l_u_0, cp_te_l_u_5, cp_te_l_u_10,
cp_te_l_u_15};

for i = 1:length(alpha)
    figure
```

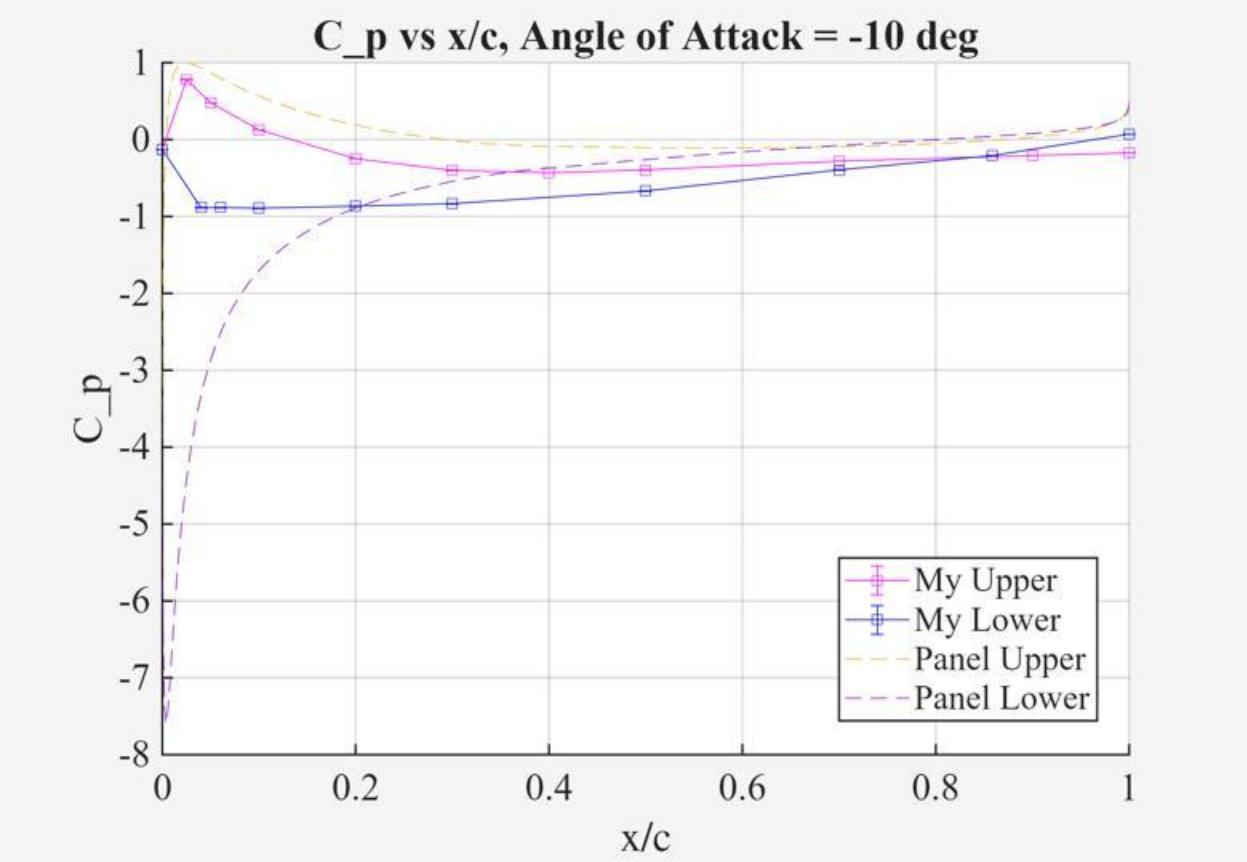
```
    hold on
    errorbar(up_x_te, cp_up{i}, cp_up_u{i}, 'ms-')
    errorbar(l_x_te,  cp_low{i}, cp_low_u{i}, 'bs-')
    size_u = min(length(xu), length(cp_u{i}));
    size_l = min(length(xl), length(cp_l{i}));
    plot(xu(1:size_u), cp_u{i}(1:size_u), '--')
    plot(xl(1:size_l), cp_l{i}(1:size_l), '--')
    grid on
    xlabel('x/c')
    ylabel('C_p')
    title(sprintf('C_p vs x/c, Angle of Attack = %g deg', alpha(i)))
    legend('My Upper','My Lower','Panel Upper','Panel Lower','Location','best')

end

% cl using pressure distrubutions
Cl_expir   = zeros(size(alpha));
CmLE = zeros(size(alpha));
CmC4 = zeros(size(alpha));
CL_u = zeros(size(alpha));

x_bar = linspace(0,1,500); % to get x/c position for both upper and lower

for i = 1:length(alpha)

    Cpu_i = interp1(up_x_te, cp_up{i}, x_bar, 'pchip','extrap');
    Cpl_i = interp1(l_x_te,  cp_low{i}, x_bar, 'pchip','extrap');

    Cl_expir(i) = trapz(x_bar, Cpl_i) - trapz(x_bar, Cpu_i);
    CmLE(i) = -(trapz(x_bar, Cpl_i .* x_bar) - trapz(x_bar, Cpu_i .* x_bar));
    CmC4(i) = -(trapz(x_bar, Cpl_i .* (x_bar - 0.25)) - trapz(x_bar, Cpu_i .* (x_bar - 0.25)));

    % uncertanity
    Cpu_i_u = interp1(up_x_te, cp_up_u{i},  x_bar, 'pchip','extrap');
    Cpl_i_u = interp1(l_x_te, cp_low_u{i},  x_bar, 'pchip','extrap');

    cl_u_u = abs((trapz(x_bar, Cpl_i) - trapz(x_bar, Cpu_i + Cpu_i_u)) - Cl_expir(i));
    cl_l_u = abs((trapz(x_bar, Cpl_i + Cpl_i_u) - trapz(x_bar, Cpu_i)) - Cl_expir(i));
    Cl_u(i) = cl_u_u + cl_l_u;

end

alpha_zl = interp1([Cl_expir(1) Cl_expir(2)], [alpha(1) alpha(2)], 0);
fprintf('Zero-lift angle of attack = %.2f deg\n', alpha_zl);

% plot cl vs alpha
figure
hold on
errorbar(alpha, Cl_expir, Cl_u, 'ms-')
plot(alpha, cl, '--')
plot(alpha_zl, 0, 'ko', 'MarkerFaceColor', 'r')
hold off
grid on
```

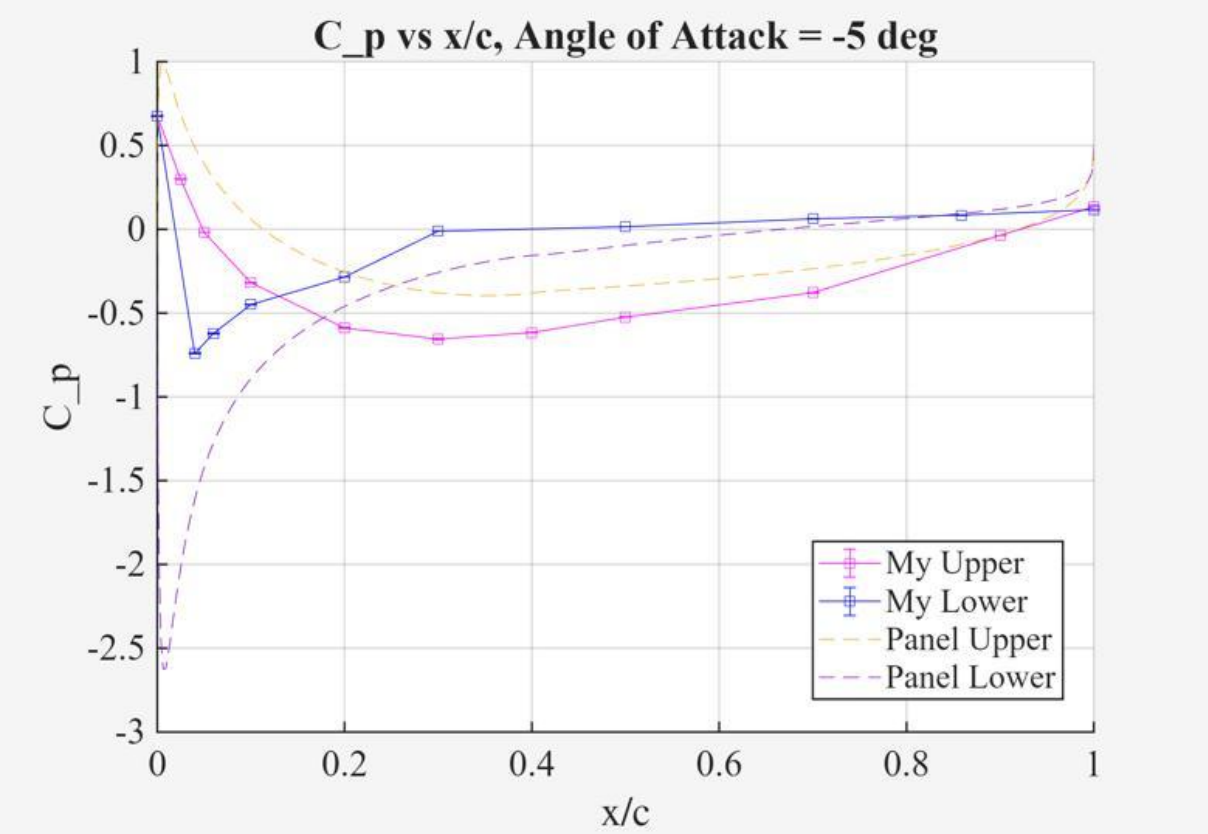
```
xlabel('alpha (deg)')
ylabel('C_l')
title('C_l vs alpha')
legend('Experiment', 'Panel', 'Location', 'best')


% cd vs alpha
Cd = zeros(size(alpha));
D_prime = zeros(size(alpha));
Cd_u = zeros(size(alpha));
D_prime_u = zeros(size(alpha));

for i = 1: length(alpha)
    vel_wake = v_mean{i}(iz1(i):iz2(i));
    pos_wake = z_m(iz1(i):iz2(i));

    vel_wake_u = v_mean_u{i}(iz1(i):iz2(i));

    D_prime(i) = rho * trapz(pos_wake, vel_wake .* (u_inf(i) - vel_wake));
    Cd(i) = D_prime(i) / (0.5 * rho * u_inf(i)^2 * c_m);

    D_u_rho = abs((rho + rho_u) * trapz(pos_wake, vel_wake .* (u_inf(i) - vel_wake)) -
D_prime(i));
    D_u_u_inf = abs((rho) * trapz(pos_wake, vel_wake .* ((u_inf(i) + u_inf_u(i)) - vel_wake)) -
D_prime(i));
    D_u_pos = abs((rho) * trapz(pos_wake + z_u, vel_wake .* (u_inf(i) - vel_wake)) - D_prime(i));
    D_u_vel_wake = abs(rho * trapz(pos_wake, (vel_wake + vel_wake_u) .* (u_inf(i) - (vel_wake +
vel_wake_u))) - D_prime(i));
    D_prime_u(i) = D_u_rho + D_u_u_inf + D_u_pos + D_u_vel_wake;

    Cd_u_D = abs((D_prime(i) + D_prime_u(i)) / (0.5 * rho * u_inf(i)^2 * c_m) - Cd(i));
    Cd_u_rho = abs(D_prime(i) / (0.5 * (rho + rho_u) * u_inf(i)^2 * c_m) - Cd(i));
    Cd_u_u_inf = abs(D_prime(i) / (0.5 * rho * (u_inf(i) + u_inf_u(i))^2 * c_m) - Cd(i));
    Cd_u(i) = Cd_u_D + Cd_u_rho + Cd_u_u_inf;

end

%disp(table(alpha(:), iz1(:), iz2(:), 'VariableNames', {'alpha','iz1','iz2'}))
%disp(table(alpha(:), Cl_expir(:), Cd(:), 'VariableNames', {'alpha','Cl','Cd'}))

figure
hold on
errorbar(alpha, Cd, Cd_u, 'ms-')
hold off
grid on
ylabel('C_d')
xlabel('alpha (deg)')
title('C_d vs alpha')

% Drag polar
figure
hold on
```

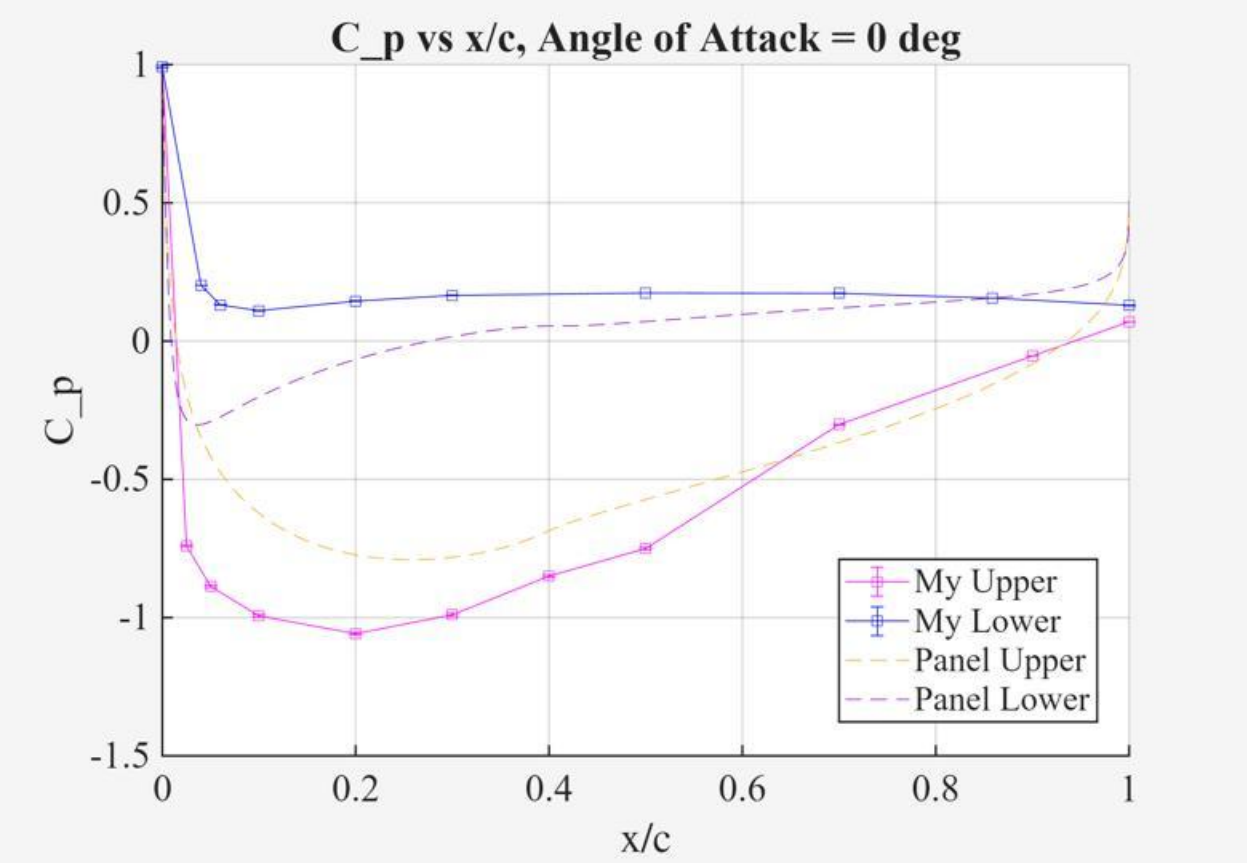
```
for i = 1:length(alpha)
    plot([Cd(i)-Cd_u(i), Cd(i)+Cd_u(i)], [Cl_expir(i), Cl_expir(i)], 'k-') % horizontal
uncertainty in Cd
    plot([Cd(i), Cd(i)], [Cl_expir(i)-Cl_u(i), Cl_expir(i)+Cl_u(i)], 'k-') % vertical uncertainty
in Cl
    plot(Cd(i), Cl_expir(i), 'o') % central point
end

plot(Cd, Cl_expir, 'b-')
hold off
grid on
xlabel('C_d')
ylabel('C_l')
title('Drag Polar')

% L/D
L_D = Cl_expir ./ Cd;

figure
hold on
plot(alpha, L_D)
hold off
grid on
xlabel('alpha')
ylabel('L/D')
title('L/D vs alpha')

% moment coefficients
figure
hold on
plot(alpha, CmLE, 'o-')
plot(alpha, cm_le, '--')
hold off
grid on
xlabel('alpha (deg)')
ylabel('C_mLE')
title('C_mLE vs alpha')
legend('Experimental','Panel','Location','best')

figure
hold on
plot(alpha, CmC4, 'o-')
plot(alpha, cm_1_4, '--')
hold off
grid on
xlabel('alpha (deg)')
ylabel('C_m_c/4')
title('C_m_c/4 vs alpha')
legend('Experimental','Panel','Location','best')
```

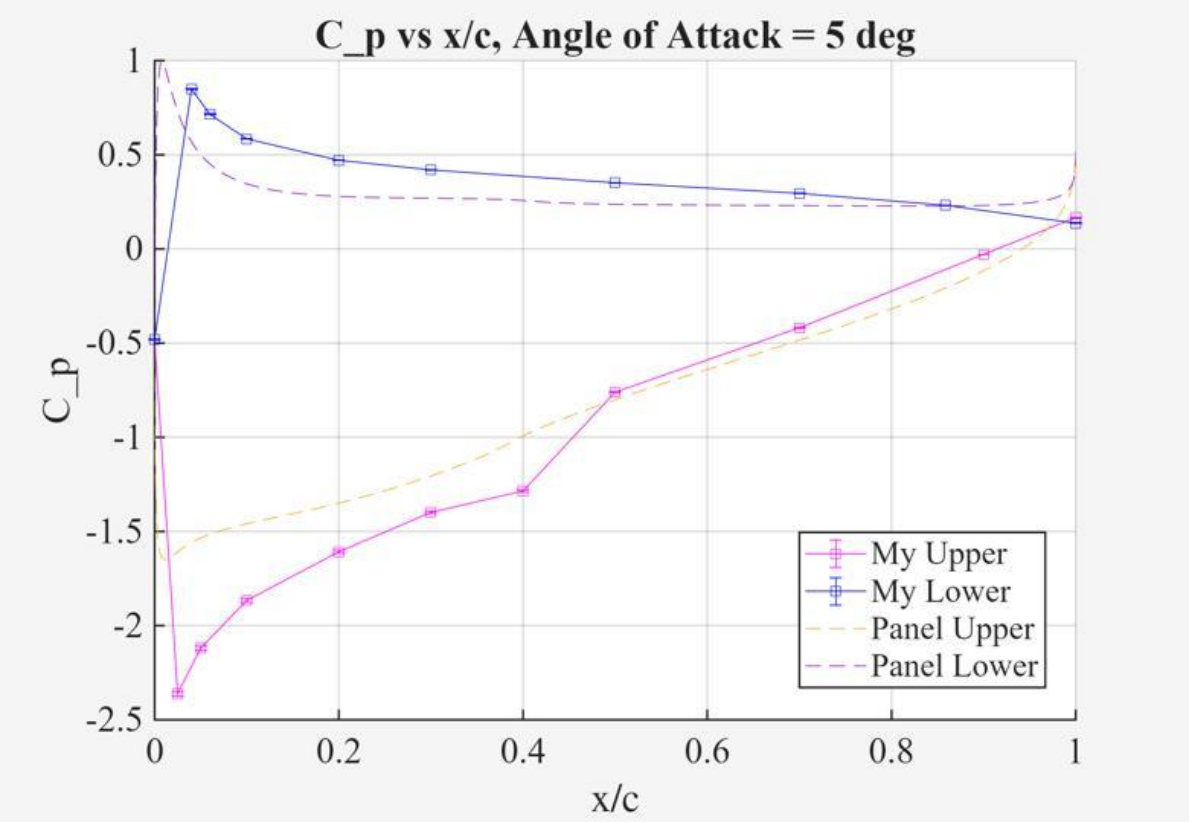
```
Zero-lift angle of attack = -7.29 deg
```

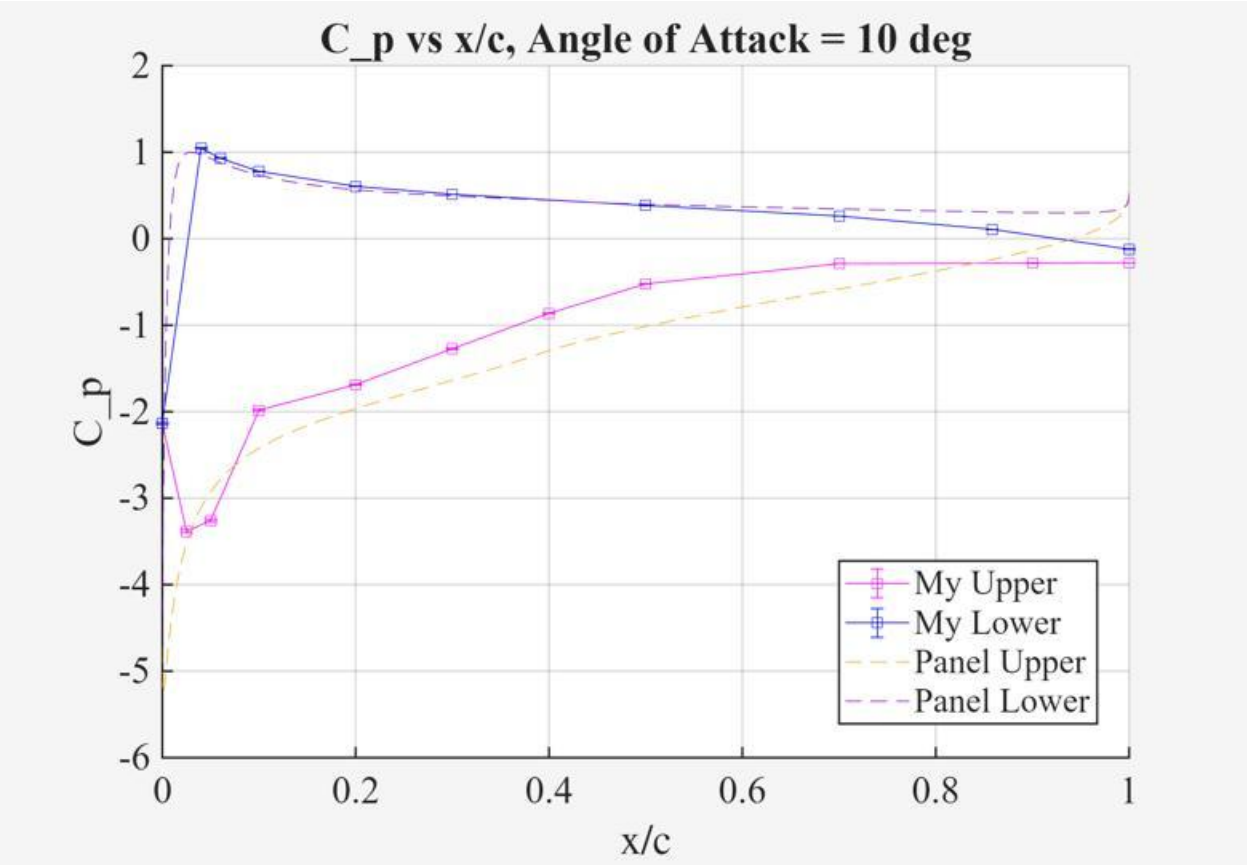

Wake Profile
Postition
Velocity (m/s)
-10°
-5°
0°
5°
10°
15°

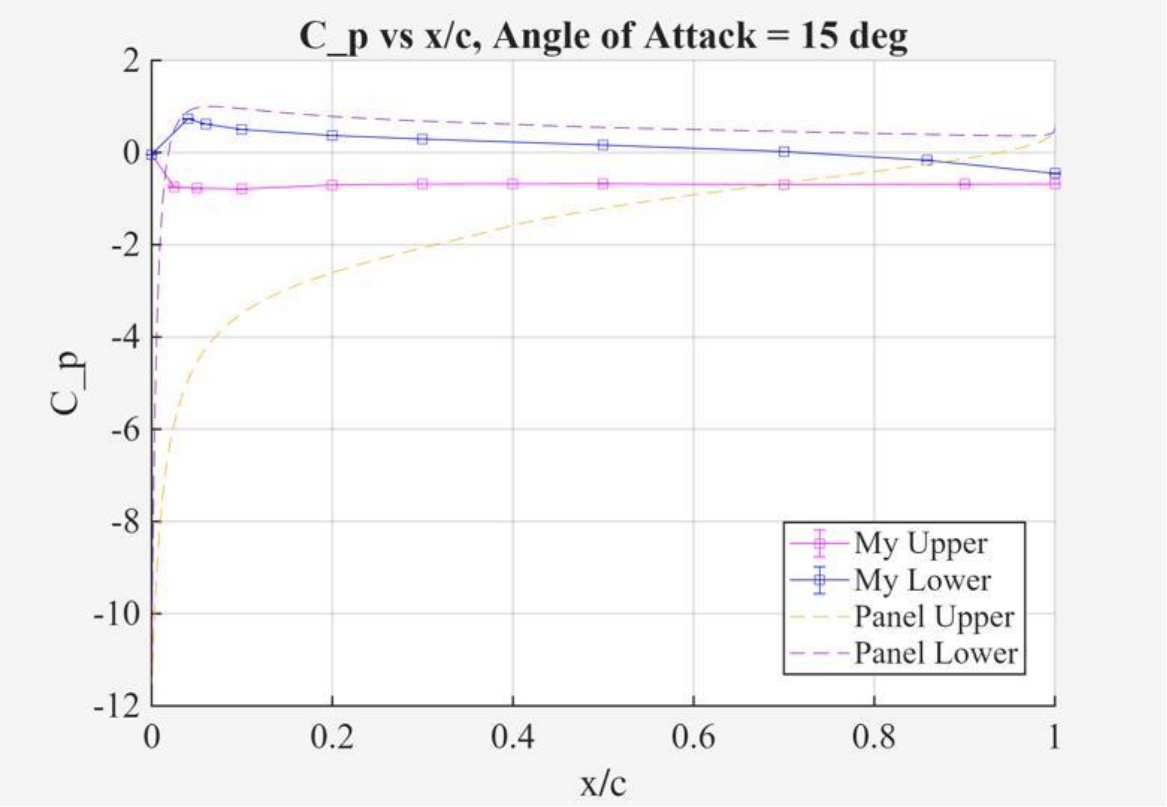

C_p vs x/c, Angle of Attack = -10 deg
C_p
x/c
My Upper
My Lower
Panel Upper
Panel Lower

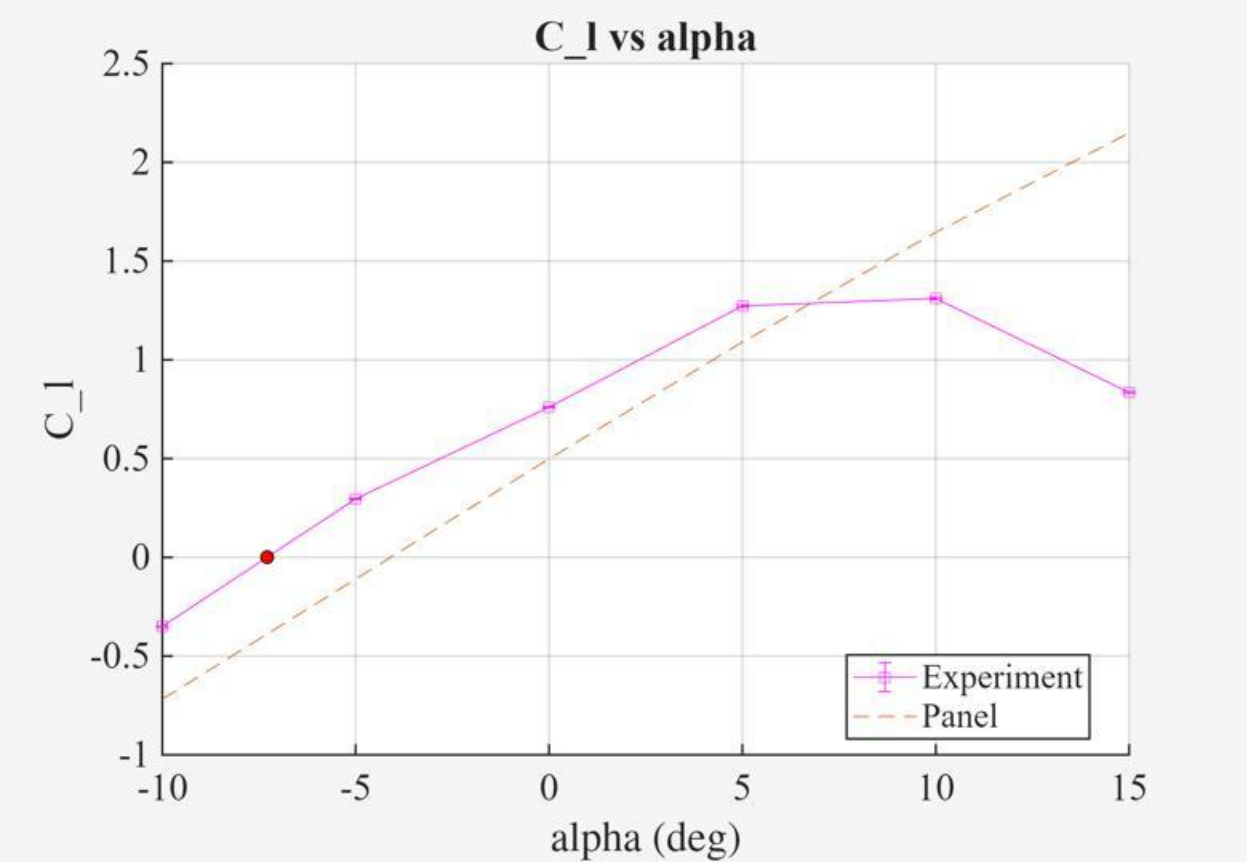

C_p vs x/c, Angle of Attack = -5 deg
C_p
x/c
My Upper
My Lower
Panel Upper
Panel Lower

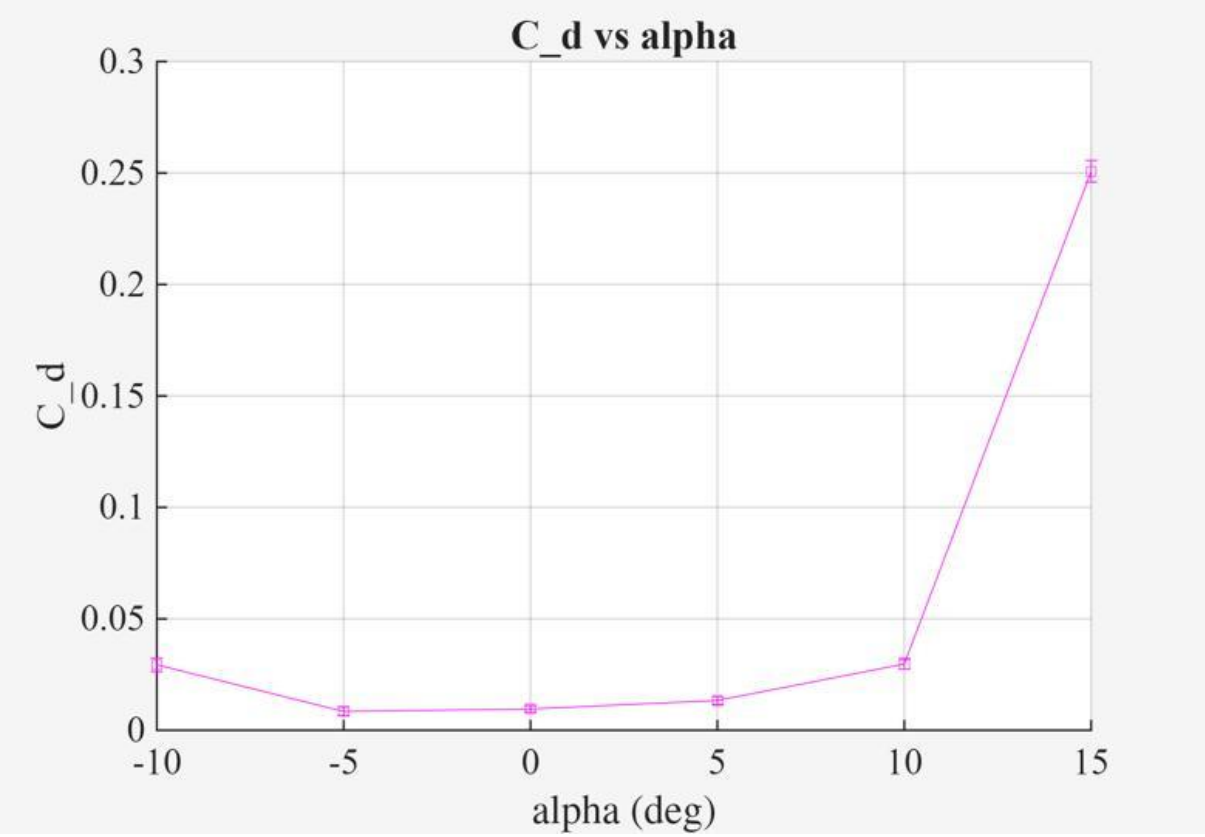

C_p vs x/c, Angle of Attack = 0 deg
C_p
x/c
My Upper
My Lower
Panel Upper
Panel Lower

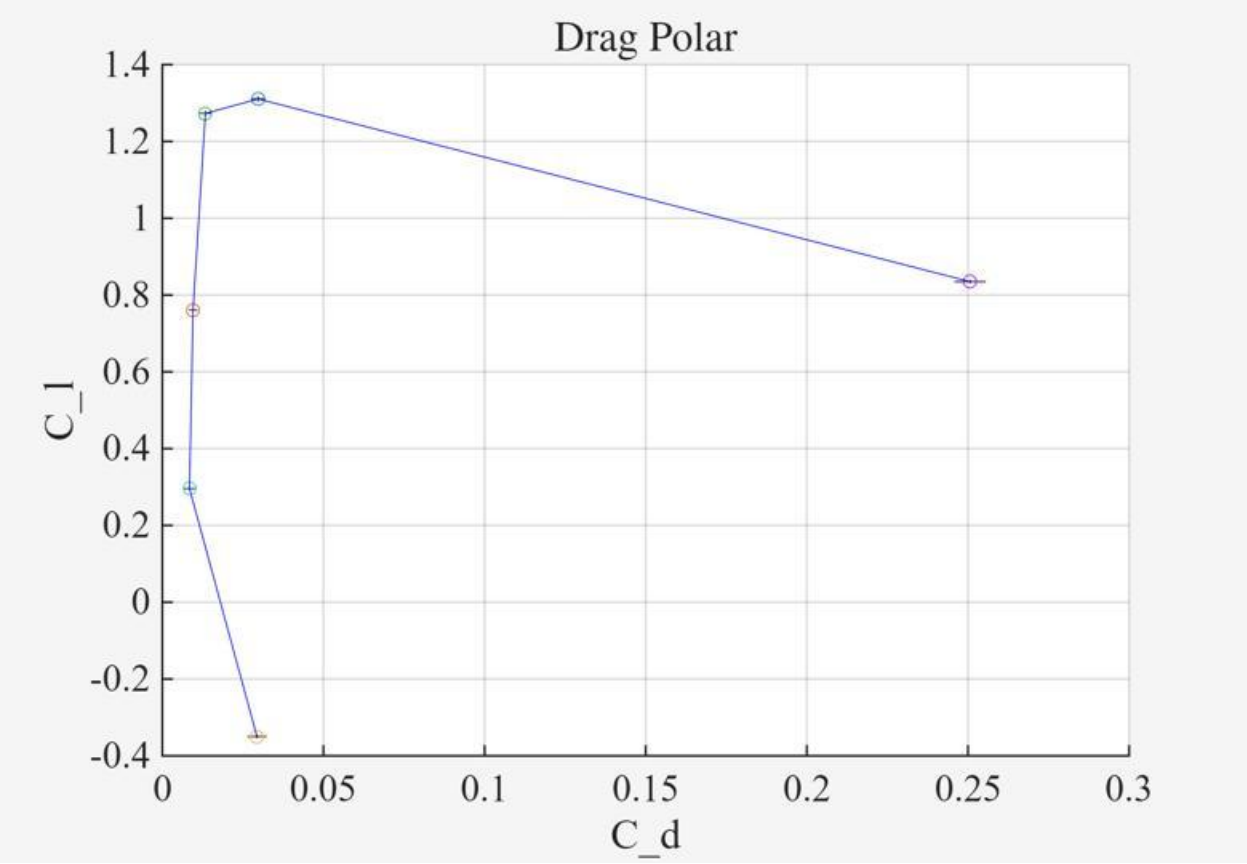

C_p vs x/c, Angle of Attack = 5 deg
C_p
x/c
1
0.5
0
-0.5
-1
-1.5
-2
-2.5
0
0.2
0.4
0.6
0.8
1
My Upper
My Lower
Panel Upper
Panel Lower

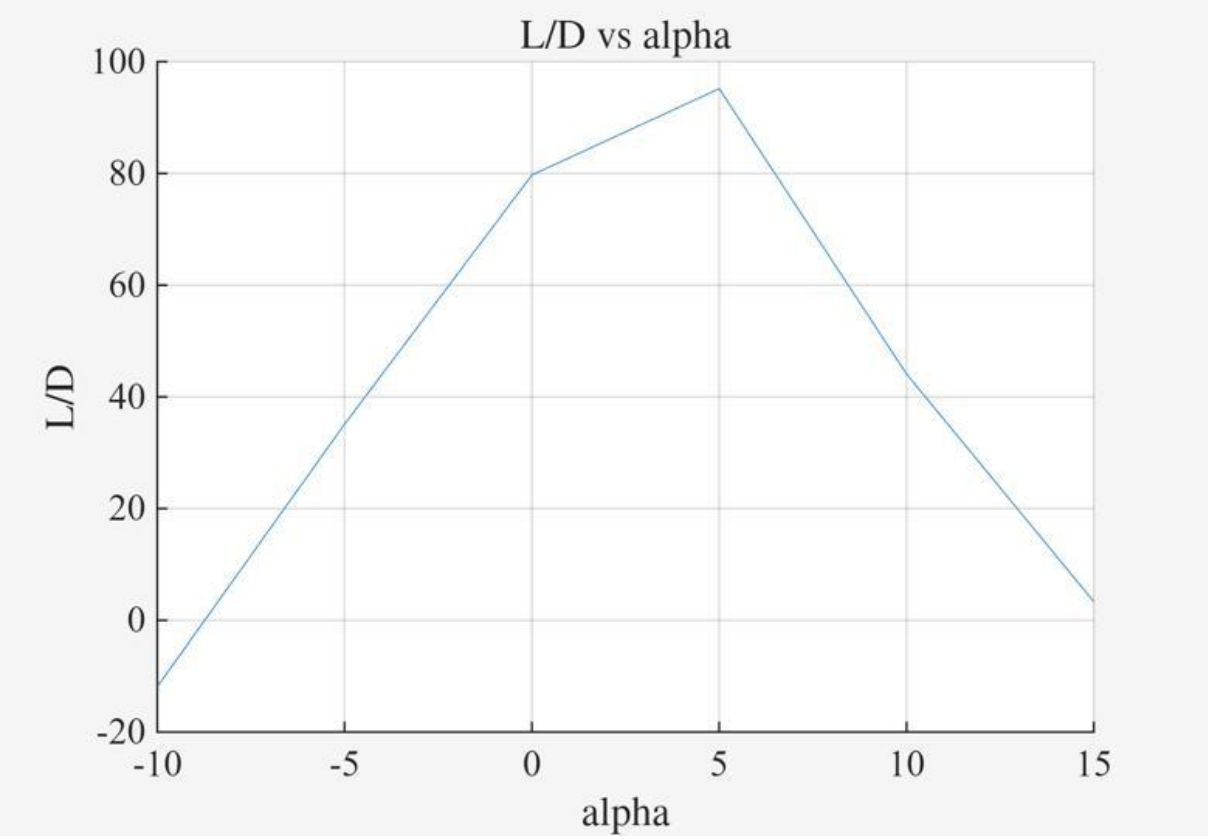

C_p vs x/c, Angle of Attack = 10 deg
C_p
x/c
2
1
0
-1
-2
-3
-4
-5
-6
0
0.2
0.4
0.6
0.8
1
My Upper
My Lower
Panel Upper
Panel Lower

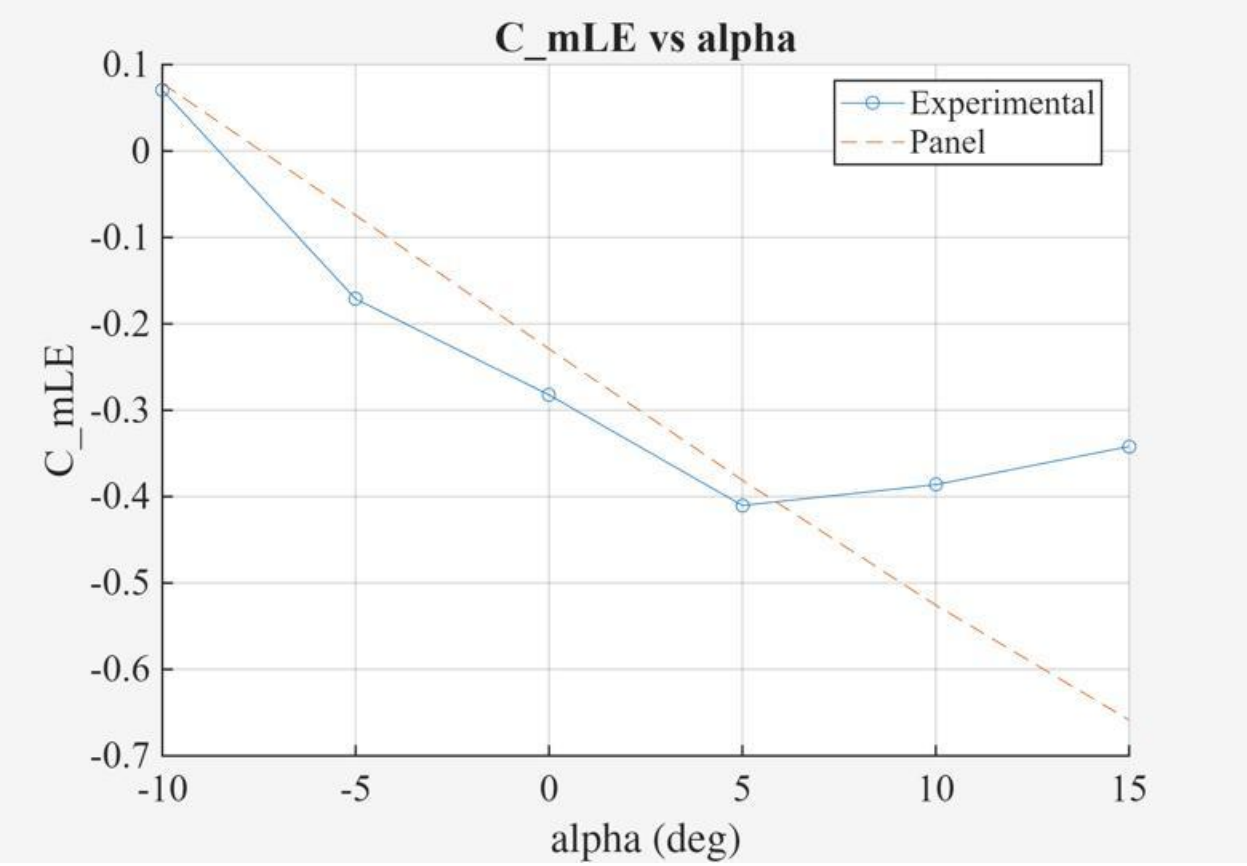

C_p vs x/c, Angle of Attack = 15 deg
C_p
x/c
My Upper
My Lower
Panel Upper
Panel Lower
2
0
-2
-4
-6
-8
-10
-12
0
0.2
0.4
0.6
0.8
1

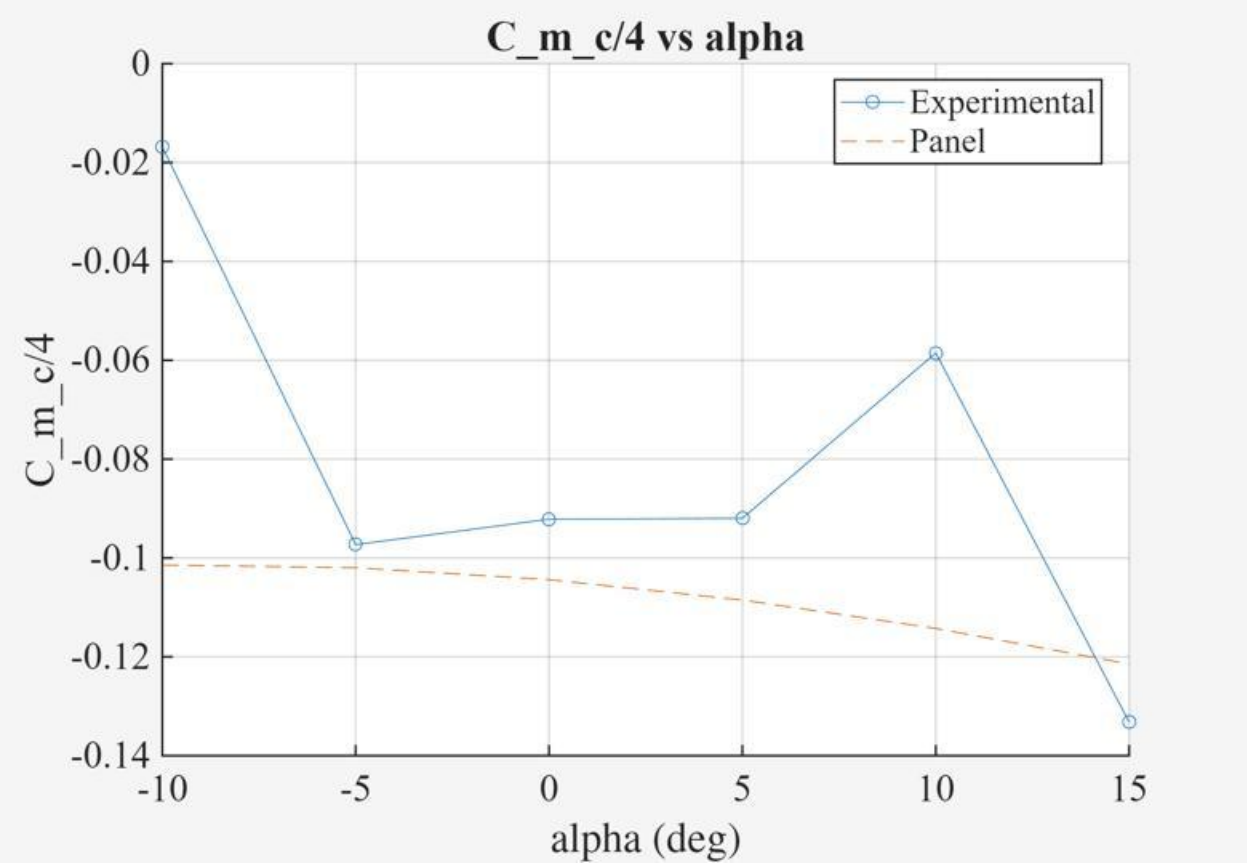

C_l vs alpha
C_l
alpha (deg)
Experiment
Panel
2.5
2
1.5
1
0.5
0
-0.5
-1
-10
-5
0
5
10
15


C_d vs alpha
C_d
alpha (deg)
0.3
0.25
0.2
0.15
0.1
0.05
0
-10
-5
0
5
10
15


Drag Polar
C_l
C_d
1.4
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
0
0.05
0.1
0.15
0.2
0.25
0.3


L/D vs alpha
L/D
alpha
100
80
60
40
20
0
-20
-10
-5
0
5
10
15


C_mLE vs alpha
C_mLE
alpha (deg)
Experimental
Panel
0.1
0
-0.1
-0.2
-0.3
-0.4
-0.5
-0.6
-0.7
-10
-5
0
5
10
15



Published with MATLAB® R2025b